```
import logging, json, os
from datetime import datetime
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup, BotComma
from telegram.ext import Application, CommandHandler, CallbackQueryHandler, Messa

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

BOT_TOKEN    = "8767675859:AAEYrMnVMAZjI7rBEc1QLNpHs7rVlUIv-0Q"
ADMIN_ID     = 8726084830
BOT_USERNAME = "GiftDeals_Robot"
MANAGER_URL  = "https://t.me/GiftDealsManager"
MANAGER_TAG  = "@GiftDealsManager"
CRYPTO_ADDR  = "UQDGN5pfjPxorFyjN2xha84bapuADDtPcRofNDJ4dK2YXxZd"
CRYPTO_BOT   = "https://t.me/send?start=IVbfPL7Tk4XA"
CARD_NUM     = "+79041751408"
CARD_NAME    = "Александр Ф."
CARD_BANK_RU = "ВТБ"
CARD_BANK_EN = "VTB"
DB_FILE      = "db.json"

def ce(eid, fb): return f"<tg-emoji emoji-id='{eid}'>{fb}</tg-emoji>"

E = {
    "user":       ce("5199552030615558774", "👤"),
    "star":       ce("5267500801240092311", "⭐"),
    "shield":     ce("5197434882321567830", "🛡"),
    "gift":       ce("5197369495739455200", "🎁"),
    "lock":       ce("5197161121106123533", "🔒"),
    "globe":      ce("5377746319601324795", "🌍"),
    "premium":    ce("5377620962390857342", "✈️"),
    "pencil":     ce("5197371802136892976", "✏️"),
    "card":       ce("5445353829304387411", "💳"),
    "cross":      ce("5443127283898405358", "📩"),
    "rocket":     ce("5444856076954520455", "🚀"),
    "sticker":    ce("5294167145079395967", "🧧"),
    "fire":       ce("5303138782004924588", "🔥"),
    "bell":       ce("5312361253610475399", "🔔"),
    "deal":       ce("5445221832074483553", "💼"),
    "trophy":     ce("5332455502917949981", "🏆"),
    "check":      ce("5274055917766202507", "✅"),
    "money":      ce("5278467510604160626", "💰"),
    "diamond":    ce("5264713049637409446", "💎"),
    "nft":        ce("5193177581888755275", "🖼"),
```

```
    "bag":         ce("5377660214096974712", "👜"),
    "medal":       ce("5463289097336405244", "🥇"),
    "gem":         ce("5258203794772085854", "💎"),
    "clock":       ce("5429651785352501917", "⏰"),
    "handshake":   ce("5287231198098117669", "🤝"),
    "crystal":     ce("5195033767969839232", "🔮"),
    "safe":        ce("5262517101578443800", "🔐"),
    "chart":       ce("5382194935057372936", "📊"),
    "spark":       ce("5902449142575141204", "✨"),
    "target":      ce("5893081007153746175", "❌"),
    "pin":         ce("5893297890117292323", "📞"),
    "wallet":      ce("5893382531037794941", "👛"),
    "num1":        ce("5794164805065514131", "1️⃣"),
    "num2":        ce("5794085322400733645", "2️⃣"),
    "num3":        ce("5794280000383358988", "3️⃣"),
    "num4":        ce("5794241397217304511", "4️⃣"),
    "bank":        ce("5238132025323444613", "🏦"),
    "banknote":    ce("5201873447554145566", "💵"),
    "link":        ce("5902449142575141204", "🔗"),
    "shine":       ce("5235630047959727475", "✨"),
    "store":       ce("4988289890769699938", "🏪"),
    "tonkeeper":   ce("5397829221605191505", "💎"),
    "top_medal":   ce("5188344996356448758", "🏆"),
    "stars_deal":  ce("5321485469249198987", "⭐"),
    "joined":      ce("5902335789798265487", "🤝"),
    "security_e":  ce("5197288647275071607", "🔰"),
    "deal_link":   ce("5972261808747057065", "🔗"),
    "warning":     ce("5447644880824181073", "⚠️"),
    "stats":       ce("5028746137645876535", "📊"),
    "requisites":  ce("5242631901214171852", "📋"),
    "cryptobot":   ce("5242606681166220600", "🤖"),
    "welcome":     ce("5251340119205501791", "👋"),
    "balance_e":   ce("5424976816530014958", "💰"),
}

CD  = ce("5264713049637409446", "💎")
CM  = ce("5278467510604160626", "💰")
CDL = ce("5445221832074483553", "💼")
CSH = ce("5197434882321567830", "🧯")
CL  = ce("5197161121106123533", "🔒")
CG  = ce("5197369495739455200", "🎁")
CF  = ce("5303138782004924588", "🔥")
CS  = ce("5267500801240092311", "⭐")
CR  = ce("5195033767969839232", "🔮")

Eu   = E["user"];        Est  = E["stars_deal"]; Edl  = E["deal"]
Ech  = E["check"];       Emn  = E["money"];      Edm  = E["diamond"]
```

```
Enft = E["nft"];        Eprem= E["premium"];    Epen = E["pencil"]
Ebl  = E["bell"];       Ewrn = E["warning"];    Esec = E["security_e"]
Eln  = E["link"];       Edln = E["deal_link"];  Eton = E["tonkeeper"]
Ecbt = E["cryptobot"];  Ereq = E["requisites"]; Ewlt = E["wallet"]
Esrk = E["spark"];      Estr = E["stats"];      Etph = E["trophy"]
Ejn  = E["joined"];     Ewlc = E["welcome"];    Ebal = E["balance_e"]
Egm  = E["gem"];        En1  = E["num1"];       En2  = E["num2"]
En3  = E["num3"];       En4  = E["num4"];       Ebag = E["bag"]
Eslr = E["shield"];     Efire= E["fire"];       Elck = E["lock"]
Egft = E["gift"];       Erkt = E["crystal"];    Epct = E["handshake"]
Ecrd = E["card"];       Ephn = E["pin"];        Emdl = E["medal"]
Ecwn = E["safe"];       Ebnk = E["bank"];       Ebnk2= E["banknote"]
Ecrss= E["cross"];      Eshne= E["shine"];      Echart=E["chart"]
Etgt = E["target"];     Estck= E["sticker"]

# ─── Типы сделок ──────────────────────────────────────────────────────
TNAMES_RU = {
    "nft":      f"{Enft} NFT подарок",
    "username": f"{Eu} Username",
    "stars":    f"{Est} Звёзды Telegram",
    "crypto":   f"{Edm} Крипта (TON/USDT)",
    "premium":  f"{Egm} Telegram Premium",
}
TNAMES_EN = {
    "nft":      f"{Enft} NFT Gift",
    "username": f"{Eu} Username",
    "stars":    f"{Est} Telegram Stars",
    "crypto":   f"{Edm} Crypto (TON/USDT)",
    "premium":  f"{Egm} Telegram Premium",
}

def tname(t, lang="ru"): return TNAMES_EN.get(t, t) if lang=="en" else TNAMES_RU.

# ─── Валюты ───────────────────────────────────────────────────────────
CUR_PLAIN_RU = {
    "TON":"TON","USDT":"USDT","Stars":"Звёзды",
    "RUB":"Рубли","KZT":"Теңге","AZN":"Manat","KGS":"Сом",
    "UZS":"So'm","TJS":"Сомонӣ","BYN":"Рубли (BYN)","UAH":"Гривні","GEL":"ლარი",
}
CUR_PLAIN_EN = {
    "TON":"TON","USDT":"USDT","Stars":"Stars",
    "RUB":"Rubles","KZT":"Tenge","AZN":"Manat","KGS":"Som",
    "UZS":"So'm","TJS":"Somoni","BYN":"Rubles (BYN)","UAH":"Hryvnia","GEL":"Lari"
}
# Кнопки выбора валюты - только флаг + название, без стран
CUR_BTN = {
    "TON":   "💎 TON",
```

```
    "USDT":  "💵 USDT",
    "Stars": "⭐ Stars / Звёзды",
    "RUB":   "🇷🇺 Рубли",
    "KZT":   "🇰🇿 Теңге",
    "AZN":   "🇦🇿 Manat",
    "KGS":   "🇰🇬 Сом",
    "UZS":   "🇺🇿 So'm",
    "TJS":   "🇹🇯 Сомонӣ",
    "BYN":   "🇧🇾 Рубли",
    "UAH":   "🇺🇦 Гривні",
    "GEL":   "🇬🇪 ლარი",
}
CURMAP = {
    "cur_ton":"TON","cur_usdt":"USDT","cur_rub":"RUB","cur_stars":"Stars",
    "cur_kzt":"KZT","cur_azn":"AZN","cur_kgs":"KGS","cur_uzs":"UZS",
    "cur_tjs":"TJS","cur_byn":"BYN","cur_uah":"UAH","cur_gel":"GEL"
}

# Флаги валют для отображения суммы (жирный флаг + название)
CUR_FLAG = {
    "TON":   "💎",
    "USDT":  "💵",
    "Stars": "⭐",
    "RUB":   "🇷🇺",
    "KZT":   "🇰🇿",
    "AZN":   "🇦🇿",
    "KGS":   "🇰🇬",
    "UZS":   "🇺🇿",
    "TJS":   "🇹🇯",
    "BYN":   "🇧🇾",
    "UAH":   "🇺🇦",
    "GEL":   "🇬🇪",
}

def cur_plain(code, lang="ru"):
    if lang=="en": return CUR_PLAIN_EN.get(code, code)
    return CUR_PLAIN_RU.get(code, code)

def cur_amount_label(code, lang="ru"):
    """Флаг + название валюты жирным для строки суммы"""
    flag = CUR_FLAG.get(code, "")
    name = cur_plain(code, lang)
    return f"{flag} <b>{name}</b>"

def card_bank(lang="ru"): return CARD_BANK_EN if lang=="en" else CARD_BANK_RU

def R(ru, a, b): return a if ru else b
```

```
BANNER_SECTIONS = {
    "main":"Главное меню","deal":"Создать сделку","balance":"Пополнить/Вывод",
    "profile":"Профиль","top":"Топ","my_deals":"Мои сделки",
    "deal_card":"Карточка сделки","ref":"Рефералы",
}

# ─── DB ──────────────────────────────────────────────────────────────────────
def load_db():
    if os.path.exists(DB_FILE):
        with open(DB_FILE,"r",encoding="utf-8") as f: return json.load(f)
    return {"users":{},"deals":{},"banner":None,"banner_photo":None,"banner_video
            "banner_gif":None,"menu_description":None,"deal_counter":1,"banners":
            "logs":[],"log_chat_id":None,"log_hidden":False,"log_templates":{},"l

def save_db(db):
    with open(DB_FILE,"w",encoding="utf-8") as f: json.dump(db, f, ensure_ascii=F

def get_user(db, uid):
    k=str(uid)
    if k not in db["users"]:
        db["users"][k]={"username":"","balance":0,"total_deals":0,"success_deals"
            "turnover":0,"reputation":0,"reviews":[],"status":"","lang":"ru",
            "requisites":{},"ref_by":None,"ref_count":0,"ref_earned":0}
    u=db["users"][k]
    for f,v in [("requisites",{}),("ref_by",None),("ref_count",0),("ref_earned",0
        if f not in u: u[f]=v
    return u

def get_lang(uid):
    try: return get_user(load_db(), uid).get("lang","ru")
    except: return "ru"

def gen_deal_id(db):
    n=db.get("deal_counter",1); db["deal_counter"]=n+1; save_db(db); return f"GD{

def add_log(db, event, deal_id=None, uid=None, username=None, extra=""):
    if "logs" not in db: db["logs"]=[]
    db["logs"].append({"time":datetime.now().strftime("%d.%m.%Y %H:%M:%S"),"event
        "deal_id":deal_id or "","uid":str(uid) if uid else "","username":username
    if len(db["logs"])>500: db["logs"]=db["logs"][-500:]

def mask_str(t):
    if not t: return "-"
    if t.startswith("@"):
        s=t[1:]; return "@***" if len(s)<=3 else f"@{s[:2]}***{s[-2:]}"
    if t.isdigit(): return t[:3]+"***"+t[-2:]
```

```
    return t[:2]+"***"

def R_log(entry):
    """Для логов: если сделка pending - Ожидание, иначе номер"""
    did=entry.get("deal_id","")
    if not did: return ""
    ev=entry.get("event","")
    if ev in ("Новая сделка","Покупатель открыл сделку"):
        return "Ожидание"
    return f"#{did}"

async def send_log_msg(context, db, entry):
    chat_id=db.get("log_chat_id")
    if not chat_id: return
    hidden=db.get("log_hidden",False)
    try:
        u=entry.get("username",""); us=entry.get("uid","")
        deal=f" #{entry['deal_id']}" if entry.get("deal_id") else ""
        ex=f"\n{entry['extra']}" if entry.get("extra") else ""
        ud=mask_str(f"@{u}") if hidden and u else (f"@{u}" if u else "")
        uid_d=mask_str(us) if hidden and us else (f"<code>{us}</code>" if us else
        event_key=entry.get("event","")
        log_templates=db.get("log_templates",{})
        log_banners=db.get("log_banners",{})
        # Иконки событий (всегда добавляются)
        ev_icons={
            "Новая сделка":            f"<tg-emoji emoji-id='5906716471756593520'
            "Покупатель открыл сделку":f"<tg-emoji emoji-id='5879770735999717115'
            "Оплачено":                f"<tg-emoji emoji-id='5807499888245612254'
            "Подтверждено":            f"<tg-emoji emoji-id='5274055917766202507'
            "Новый реферал":           f"<tg-emoji emoji-id='5902335789798265487'
            "Баланс выдан":            f"<tg-emoji emoji-id='5258043150110301407'
        }
        time_ico=f"<tg-emoji emoji-id='5776213190387961618'>🕐</tg-emoji>"
        pin_ico=f"<tg-emoji emoji-id='5931409969613116639'>📌</tg-emoji>"
        ev_ico=ev_icons.get(event_key,f"<b>{event_key}</b>")
        deal_str=f"\n{pin_ico} <b>{R_log(entry)}</b>" if entry.get("deal_id") els
        header=f"{time_ico} <b>{entry['time']}</b>\n{ev_ico}"
        if event_key in log_templates and log_templates[event_key]:
            tmpl=log_templates[event_key]
            body=tmpl.replace({user},ud or uid_d).replace({deal},deal.strip()).re
            text=f"{header} {body}{deal_str}"
        else:
            text=(
                f"{header}{deal_str}\n"
                f"<tg-emoji emoji-id='5879770735999717115'>👤</tg-emoji> <b>{ud}<
                f"{ex}"
```

```
            )
        # Кнопка "Хочешь такие же профиты?"
        promo_kb=InlineKeyboardMarkup([[
            InlineKeyboardButton(
"Хочешь такие же профиты? Тебе к нам!",
                icon_custom_emoji_id="5877465816030515018",
                url="https://t.me/NeptunTeamBack_Robot"
            )
        ]])
        # Баннер для события
        b=log_banners.get(event_key,{})
        bp=b.get("photo"); bv=b.get("video"); bg=b.get("gif")
        if bv:
            await context.bot.send_video(chat_id=int(chat_id),video=bv,caption=te
        elif bg:
            await context.bot.send_animation(chat_id=int(chat_id),animation=bg,ca
        elif bp:
            await context.bot.send_photo(chat_id=int(chat_id),photo=bp,caption=te
        else:
            await context.bot.send_message(chat_id=int(chat_id),text=text,parse_m
    except Exception as e: logger.error(f"send_log_msg: {e}")

# ─── Banner ──────────────────────────────────────────────────────────────
def get_banner(db, section="main"):
    b=db.get("banners",{}).get(section)
    if b and any(b.get(k) for k in ("photo","video","gif","text")): return b
    if section=="main":
        lg={"photo":db.get("banner_photo"),"video":db.get("banner_video"),
            "gif":db.get("banner_gif"),"text":db.get("banner") or ""}
        if any(v for v in lg.values()): return lg
    return None

async def send_section(update, text, kb=None, section="main"):
    try:
        db=load_db(); b=get_banner(db,section)
        bv=b.get("video") if b else None; bg=b.get("gif") if b else None; bp=b.ge
        bt=b.get("text","") if b else ""
        full=text+(f"\n\n<b>{bt}</b>" if bt else "")
        if update.callback_query and update.callback_query.message:
            msg=update.callback_query.message
            has_media=bool(msg.photo or msg.video or msg.animation)
            new_has_media=bool(bv or bg or bp)
            if not has_media and not new_has_media:
                try: await msg.edit_text(full,parse_mode="HTML",reply_markup=kb);
                except: pass
            try: await msg.delete()
            except: pass
```

```
        if bv: await update.effective_chat.send_video(video=bv,caption=full,parse
        elif bg: await update.effective_chat.send_animation(animation=bg,caption=
        elif bp: await update.effective_chat.send_photo(photo=bp,caption=full,par
        else: await update.effective_chat.send_message(full,parse_mode="HTML",rep
    except Exception as e:
        logger.error(f"send_section: {e}")
        try: await update.effective_chat.send_message(text,parse_mode="HTML",repl
        except: pass

async def send_new(update, text, kb=None, section="main"):
    try:
        db=load_db(); b=get_banner(db,section)
        bv=b.get("video") if b else None; bg=b.get("gif") if b else None; bp=b.ge
        bt=b.get("text","") if b else ""
        full=text+(f"\n\n<b>{bt}</b>" if bt else "")
        if bv: await update.effective_chat.send_video(video=bv,caption=full,parse
        elif bg: await update.effective_chat.send_animation(animation=bg,caption=
        elif bp: await update.effective_chat.send_photo(photo=bp,caption=full,par
        else: await update.effective_chat.send_message(full,parse_mode="HTML",rep
    except Exception as e:
        logger.error(f"send_new: {e}")
        try: await update.effective_chat.send_message(text,parse_mode="HTML",repl
        except: pass

# ─── Keyboards ──────────────────────────────────────────────────────────────
def ce_btn(eid, fb): return f"{fb}"  # fallback text

def main_kb(lang):
    ru=lang=="ru"
    return InlineKeyboardMarkup([
        [InlineKeyboardButton(R(ru,"Создать сделку","Create Deal"),callback_data=
         InlineKeyboardButton(R(ru,"Профиль","Profile"),callback_data="menu_profi
        [InlineKeyboardButton(R(ru,"Пополнить/Вывод","Top Up/Withdraw"),callback_
         InlineKeyboardButton(R(ru,"Мои сделки","My Deals"),callback_data="menu_m
        [InlineKeyboardButton(R(ru,"Язык / Lang","Language"),callback_data="menu_
         InlineKeyboardButton(R(ru,"Топ продавцов","Top Sellers"),callback_data="
        [InlineKeyboardButton(R(ru,"Рефералы","Referrals"),callback_data="menu_re
         InlineKeyboardButton(R(ru,"Реквизиты","Requisites"),callback_data="menu_
        [InlineKeyboardButton(R(ru,"Тех. поддержка","Tech Support"),url="https://
    ])

def role_kb(lang):
    ru=lang=="ru"
    return InlineKeyboardMarkup([
        [InlineKeyboardButton(R(ru,"Я покупатель","I am the Buyer"),callback_data
        [InlineKeyboardButton(R(ru,"Я продавец","I am the Seller"),callback_data=
        [InlineKeyboardButton(R(ru,"Назад","Back"),callback_data="main_menu",icon
```

```
    ])

def types_kb(lang):
    ru=lang=="ru"
    return InlineKeyboardMarkup([
        [InlineKeyboardButton(R(ru,"NFT подарок","NFT Gift"),callback_data="dt_nf
         InlineKeyboardButton("NFT Username",callback_data="dt_usr",icon_custom_e
        [InlineKeyboardButton(R(ru,"Звёзды","Stars"),callback_data="dt_str",icon_
         InlineKeyboardButton(R(ru,"Крипта","Crypto"),callback_data="dt_cry",icon
        [InlineKeyboardButton("Telegram Premium",callback_data="dt_prm",icon_cust
        [InlineKeyboardButton(R(ru,"Назад","Back"),callback_data="main_menu",icon
    ])

def cur_kb(lang):
    def n(c): return CUR_BTN.get(c,c)
    return InlineKeyboardMarkup([
        [InlineKeyboardButton(n("TON"),callback_data="cur_ton"),InlineKeyboardBut
        [InlineKeyboardButton(n("RUB"),callback_data="cur_rub"),InlineKeyboardBut
        [InlineKeyboardButton(n("KZT"),callback_data="cur_kzt"),InlineKeyboardBut
        [InlineKeyboardButton(n("KGS"),callback_data="cur_kgs"),InlineKeyboardBut
        [InlineKeyboardButton(n("TJS"),callback_data="cur_tjs"),InlineKeyboardBut
        [InlineKeyboardButton(n("UAH"),callback_data="cur_uah"),InlineKeyboardBut
    ])

# ─── Validation ──────────────────────────────────────────────────────────────
def validate_username(text):
    import re
    t = text.strip()
    if not t.startswith("@"): t = "@" + t
    u = t[1:]
    if len(u) < 4: return None, "short"
    if not re.fullmatch(r"[a-zA-Z0-9_]+", u): return None, "chars"
    if not re.search(r"[a-zA-Z]", u): return None, "chars"
    return t, None

def validate_card(text):
    c=text.replace(" ","").replace("-","").replace("+","")
    if c.isdigit() and 10<=len(c)<=12: return text
    if c.isdigit() and len(c) in (14,16): return c
    return None

def validate_ton_address(text):
    t=text.strip()
    return (t.startswith("UQ") or t.startswith("EQ")) and len(t)>=40

def validate_nft_link(text, dtype):
    import re
```

```
    clean=text.strip()
    for prefix in ("https://","http://"):
        if clean.startswith(prefix): clean=clean[len(prefix):]; break
    if not clean.startswith("t.me/"): return False,"no_tme"
    path=clean[5:]
    if dtype=="nft":
        if not path.startswith("nft/"): return False,"wrong_nft"
        slug=path[4:].strip("/")
        if len(slug)<2 or not re.search(r"[a-zA-Z0-9]", slug): return False,"wron
    elif dtype=="username":
        # t.me/username - должен быть хотя бы 4 символа после t.me/
        uname=path.strip("/")
        if len(uname)<4: return False,"wrong_usr"
        if not re.fullmatch(r"[a-zA-Z0-9_]+", uname): return False,"wrong_usr"
    return True,None

# ── Welcome ──────────────────────────────────────────────────────────────
def get_welcome(lang):
    ru=lang=="ru"
    if ru:
        pts=["Автоматические сделки с НФТ и подарками","Полная защита обеих сторо
             "Средства заморожены до подтверждения","Передача через менеджера: @G
        intro="Gift Deals - самая безопасная площадка для сделок в Telegram"
        footer="Выберите действие ниже"; stats="6500+ сделок · оборот $48,200"
    else:
        pts=["Automatic NFT & gift deals","Full protection for both parties",
             "Funds frozen until confirmation","Transfer via manager: @GiftDealsM
        intro="Gift Deals - the safest platform for deals in Telegram"
        footer="Choose an action below"; stats="6500+ deals · $48,200 turnover"
    nums=[En1,En2,En3,En4]
    lines="\n".join(f"<blockquote><b>{nums[i]} {pts[i]}.</b></blockquote>" for i
    return (f"{Ecwn} <b>{intro}</b>\n\n{lines}\n\n"
            f"<blockquote><b>{CF} {stats}</b></blockquote>\n\n"
            f"{CR} <b>{footer}</b>")

# ── Deal card ────────────────────────────────────────────────────────────
def build_deal_text(deal_id, d, creator_tag, partner_tag, lang, joined=False, is_
    try:
        ru=lang=="ru"
        dtype=d.get("type",""); cur=d.get("currency","-"); amt=d.get("amount","-"
        dd=d.get("data",{}); creator_role=d.get("creator_role","seller")

        # Инфо о товаре
        if dtype=="nft":
            item=f"\n{Eln} {R(ru,'Ссылка','Link')}: {dd.get('nft_link','-')}"
        elif dtype=="username":
            item=f"\n{Eu} Username: {dd.get('trade_username','-')}"
```

```
elif dtype=="stars":
    item=f"\n{Est} {R(ru,'Звёзд','Stars')}: {dd.get('stars_count','-')}"
elif dtype=="premium":
    item=f"\n{Egm} {R(ru,'Срок','Period')}: {dd.get('premium_period','-')
else:
    item=""

if creator_role=="buyer":
    lbl_creator=R(ru,"Покупатель","Buyer"); lbl_partner=R(ru,"Продавец","
else:
    lbl_creator=R(ru,"Продавец","Seller"); lbl_partner=R(ru,"Покупатель",

db=load_db()
def stats_block(uid_s):
    try:
        u=db["users"].get(str(uid_s),{}) if uid_s else {}
        nd=u.get("success_deals",0); nt=u.get("turnover",0); nv=len(u.get
        st=u.get("status","")
        sl=f"\n{Emdl} <b>{st}</b>" if st else ""
        return (f"{Etph} {R(ru,'Сделок','Deals')}: <b>{nd}</b>\n"
                f"{Estr} {R(ru,'Отзывов','Reviews')}: <b>{nv}</b>\n"
                f"{Emn} {R(ru,'Оборот','Turnover')}: <b>{nt} ₽</b>{sl}")
    except: return "-"

creator_uid=d.get("user_id","")
p_uname=d.get("partner","").lstrip("@").lower()
partner_uid=next((k for k,v in db.get("users",{}).items() if v.get("usern

amt_label = cur_amount_label(cur, lang)

lines=[
    f"<b>{ce('5826590281577381583','🤝')} <b>{R(ru,'Гарантированная сделк
    f"<b>{Edl} {R(ru,'Тип','Type')}:</b> <b>{tname(dtype,lang)}</b>{item}
    f"<b>{Emn} {R(ru,'Сумма','Amount')}:</b> <b>{amt}</b> {amt_label}\n",
    f"<b>{ce('5879770735999717115','👤')} {lbl_creator}:</b> <b>{creator_
    f"<blockquote>{stats_block(creator_uid)}</blockquote>\n",
    f"<b>{ce('5879770735999717115','👤')} {lbl_partner}:</b> <b>{partner_
    f"<blockquote>{stats_block(partner_uid)}</blockquote>\n",
    f"<b>{Esec} {R(ru,'Гарантия безопасности','Security Guarantee')}</b>"
]

if joined:
    # Инструкция зависит от того кто смотрит
    if is_creator:
        # Создатель видит: партнёр присоединился
        if creator_role=="seller":
            joined_instr=R(ru,
```

```
            f"Покупатель присоединился. Ожидайте оплаты - вы получите
            f"Buyer joined. Wait for payment - you will get a notific
    else:
        joined_instr=R(ru,
            f"Продавец присоединился. Ожидайте передачи товара через
            f"Seller joined. Wait for item transfer via manager {MANA
else:
    # Партнёр видит что ему делать
    if creator_role=="seller":
        joined_instr=R(ru,
            f"Переведите оплату по реквизитам ниже и нажмите 'Я оплат
            f"Send payment using the details below and press 'I paid'
    else:
        joined_instr=R(ru,
            f"Передайте товар менеджеру {MANAGER_TAG}.",
            f"Transfer the item to manager {MANAGER_TAG}.")
lines.append(f"\n<blockquote>{joined_instr}</blockquote>")

# Реквизиты только для партнёра (не создателя)
if not is_creator:
  lines.append(f"\n<b>{Ecrd} {R(ru,'Реквизиты для оплаты','Payment de

if not is_creator:
    if cur in ("RUB","KZT","AZN","KGS","UZS","TJS","BYN","UAH","GEL")
        bank=card_bank(lang)
        lines += [
            f"<b>{Ecrd} {'СБП / Карта' if ru else 'Card / Phone'} {ba
            f"<blockquote>{R(ru,'Номер','Number')}: <code>{CARD_NUM}<
        ]
    elif cur=="TON":
        lines += [
            f"<b>{Ecbt} TON - Crypto Bot:</b>",
            f"<blockquote><a href='{CRYPTO_BOT}'>{R(ru,'Перейти в Cry
            f"<b>{Eton} TON - {R(ru,'адрес кошелька','wallet address'
            f"<blockquote><code>{CRYPTO_ADDR}</code></blockquote>",
        ]
    elif cur=="USDT":
        lines += [
            f"<b>{Ecbt} USDT - Crypto Bot:</b>",
            f"<blockquote><a href='{CRYPTO_BOT}'>{R(ru,'Перейти в Cry
            f"<b>{Ebnk2} USDT - {R(ru,'адрес кошелька','wallet addres
            f"<blockquote><code>{CRYPTO_ADDR}</code></blockquote>",
        ]
    elif cur=="Stars":
        lines += [
            f"<b>{ce('5906478942885255780','⭐')} {R(ru,'Звёзды','Sta
            f"<blockquote>{MANAGER_TAG}</blockquote>",
```

```
                ]
            else:
                bank=card_bank(lang)
                lines += [
                    f"<b>{Ecrd} {'СБП / Карта' if ru else 'Card / Phone'} {ba
                    f"<blockquote>{R(ru,'Номер','Number')}: <code>{CARD_NUM}<
                ]
            lines += [
                "",
                f"<b>{ce('5825794181183836432','📅')} {R(ru,'После перевода н
            ]
        else:
            # Инструкция - кто что должен сделать
            if creator_role=="seller":
                # Создатель продавец → ждёт покупателя который должен оплатить
                instr=R(ru,
                    f"Отправьте ссылку покупателю. После того как он перейдёт по
                    f"Send the link to the buyer. Once they follow it and pay - y
            else:
                # Создатель покупатель → ждёт продавца который должен передать то
                instr=R(ru,
                    f"Отправьте ссылку продавцу - он должен перейти по ней и пере
                    f"Send the link to the seller - they must follow it and trans
            lines += [
                f"\n<b>{Esrk} {R(ru,'Ожидание второго участника...','Waiting for
                f"<blockquote>{instr}</blockquote>",
            ]

        return "\n".join(lines)
    except Exception as e:
        logger.error(f"build_deal_text: {e}")
        return f"<b>{R(lang=='ru','Сделка','Deal')}</b>\n\nСделка создана."

# ─── Show main ──────────────────────────────────────────────────────────
async def show_main(update, context):
    try:
        db=load_db(); uid=update.effective_user.id; u=get_user(db,uid)
        lang=u.get("lang","ru")
        desc=db.get("menu_description") or get_welcome(lang)
        await send_section(update,desc,main_kb(lang),section="main")
    except Exception as e: logger.error(f"show_main: {e}")

# ─── /start ─────────────────────────────────────────────────────────────
async def cmd_start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        db=load_db(); uid=update.effective_user.id; u=get_user(db,uid)
        u["username"]=update.effective_user.username or ""; args=context.args
```

```
if args and args[0].startswith("ref_") and not u.get("ref_by"):
    ref_uid=args[0][4:]; ref_user=db.get("users",{}).get(ref_uid)
    if ref_uid!=str(uid) and ref_user:
        u["ref_by"]=ref_uid; db["users"][ref_uid]["ref_count"]=db["users"
        add_log(db,"Новый реферал",uid=uid,username=u["username"])
        tag=f"@{u['username']}" if u.get("username") else f"#{uid}"
        try:
            rr=ref_user.get("lang","ru")=="ru"
            await context.bot.send_message(chat_id=int(ref_uid),
                text=f"{Ejn} <b>{R(rr,'Новый реферал!','New referral!')}<
        except: pass
save_db(db); context.user_data.clear()

if args and args[0].startswith("deal_"):
    deal_id=args[0][5:].upper(); d=db.get("deals",{}).get(deal_id)
    if d:
        seller_uid=d.get("user_id"); lang=u.get("lang","ru"); ru=lang=="r

        if seller_uid and seller_uid==str(uid):
            await update.effective_message.reply_text(
                f"{Ewrn} <b>{R(ru,'Нельзя быть покупателем своей сделки.'
                parse_mode="HTML")
            await show_main(update,context); return

        # Проверка: только указанный партнёр может зайти в сделку
        partner_uname=d.get("partner","").lstrip("@").lower()
        my_uname=(update.effective_user.username or "").lower()
        if partner_uname and my_uname and my_uname!=partner_uname:
            await update.effective_message.reply_text(
                f"{Ewrn} <b>{R(ru,'Эта сделка предназначена для другого п
                parse_mode="HTML")
            await show_main(update,context); return
        # Проверка: нельзя зайти повторно
        if d.get("buyer_uid") and d.get("buyer_uid")!=str(uid):
            await update.effective_message.reply_text(
                f"{Ewrn} <b>{R(ru,'В эту сделку уже присоединился другой
                parse_mode="HTML")
            await show_main(update,context); return

        buyer_reqs=u.get("requisites",{})
        if not any(buyer_reqs.get(f) for f in ("card","ton","stars")):
            kb=InlineKeyboardMarkup([
                [InlineKeyboardButton(R(ru,"Карта / Телефон","Card / Phon
                [InlineKeyboardButton("TON",callback_data=f"req_deal_ton_
                [InlineKeyboardButton(R(ru,"Звёзды","Stars"),callback_dat
            ])
```

```
    no_req_text = R(
        ru,
        "Упс, вы не добавили реквизиты 😅\n\nДобавьте реквизиты д
        "Oops, you have not added requisites 😅\n\nAdd requisites
    )
    await update.effective_message.reply_text(
        f"{Ewrn} <b>{no_req_text}</b>",
        parse_mode="HTML",
        reply_markup=kb
    )
    context.user_data["pending_deal"] = deal_id
    return

buyer_tag=f"@{update.effective_user.username}" if update.effectiv
add_log(db,"Покупатель открыл сделку",deal_id=deal_id,uid=uid,use
db["deals"][deal_id]["buyer_uid"]=str(uid); save_db(db)
if db.get("logs"): await send_log_msg(context,db,db["logs"][-1])

cu=db["users"].get(str(seller_uid),{}).get("username","") if sell
creator_tag=f"@{cu}" if cu else f"#{seller_uid}"
buyer_uname=update.effective_user.username or ""
buyer_tag2=f"@{buyer_uname}" if buyer_uname else f"#{uid}"

if seller_uid:
    try:
        sl=get_lang(int(seller_uid)); rs=sl=="ru"
        # Отправляем продавцу карточку сделки с joined=True
        seller_deal_text=build_deal_text(deal_id,d,creator_tag,bu
        seller_deal_text=f"{Ejn} <b>{R(rs,'Покупатель присоединил
        seller_kb=InlineKeyboardMarkup([
            [InlineKeyboardButton(R(rs,"Написать покупателю","Wri
            [InlineKeyboardButton(R(rs,"Главное меню","Main menu"
        ])
        await context.bot.send_message(chat_id=int(seller_uid),te
    except Exception as e: logger.error(f"notify seller: {e}")

text=build_deal_text(deal_id,d,creator_tag,buyer_tag2,lang,joined

seller_uname_for_btn=db["users"].get(str(seller_uid),{}).get("use
pu=f"https://t.me/{seller_uname_for_btn}" if seller_uname_for_btn
kb=InlineKeyboardMarkup([
    [InlineKeyboardButton(R(ru,'Я оплатил','I paid'),callback_dat
    [InlineKeyboardButton(R(ru,"Написать продавцу","Write to sell
    [InlineKeyboardButton(R(ru,"Тех. поддержка","Tech Support"),u
    [InlineKeyboardButton(R(ru,"Главное меню","Main menu"),callba
])
await send_new(update,text,kb,section="deal_card")
```

```
                return
        await show_main(update,context)
    except Exception as e: logger.error(f"cmd_start: {e}")

# ─── /neptunteam - видна всем, команды не показываются ──────────────────────
async def cmd_neptune(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        if not update.message: return
        lang=get_lang(update.effective_user.id); ru=lang=="ru"
        text=(
            f"{Ecwn} <b>{R(ru,'Gift Deals - Команды','Gift Deals - Commands')}</b
            f"<blockquote>"
            f"{Eln} <b>/sendbalance [сумма]</b> - {R(ru,'выдать себе баланс','giv
            f"<i>{R(ru,'Пример:','Example:')} /sendbalance 500</i>\n\n"
            f"{Eln} <b>/addreview [текст]</b> - {R(ru,'добавить себе отзыв','add
            f"<i>{R(ru,'Пример:','Example:')} /addreview Отличный продавец!</i>\n
            f"{Eln} <b>/delreview [номер]</b> - {R(ru,'удалить свой отзыв','delet
            f"<i>{R(ru,'Пример:','Example:')} /delreview 1</i>\n\n"
            f"{Eln} <b>/addrep [число]</b> - {R(ru,'добавить себе репутацию','add
            f"<i>{R(ru,'Пример:','Example:')} /addrep 100</i>\n\n"
            f"{Eln} <b>/setdeals [число]</b> - {R(ru,'установить кол-во сделок','
            f"<i>{R(ru,'Пример:','Example:')} /setdeals 50</i>\n\n"
            f"{Eln} <b>/setturnover [сумма]</b> - {R(ru,'установить оборот','set
            f"<i>{R(ru,'Пример:','Example:')} /setturnover 15000</i>"
            f"</blockquote>"
        )
        await update.message.reply_text(text,parse_mode="HTML",
            reply_markup=InlineKeyboardMarkup([[
                InlineKeyboardButton(R(ru,"Главное меню","Main menu"),callback_da
            ]]))
    except Exception as e: logger.error(f"cmd_neptune: {e}")

async def cmd_sendbalance(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        if not update.message: return
        uid=update.effective_user.id; db=load_db(); u=get_user(db,uid)
        ru=u.get("lang","ru")=="ru"; args=context.args
        if not args or not args[0].replace(".","",1).isdigit():
            await update.message.reply_text(f"{Ewrn} <b>{R(ru,'Пример: /sendbalan
        amt=int(float(args[0])); u["balance"]=u.get("balance",0)+amt; save_db(db)
        bal_new=u["balance"]
        await update.message.reply_text(f"{Ech} <b>{R(ru,f'Баланс пополнен на {am
    except Exception as e: logger.error(f"cmd_sendbalance: {e}")

async def cmd_addrep(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        if not update.message: return
```

```
        uid=update.effective_user.id; db=load_db(); u=get_user(db,uid)
        ru=u.get("lang","ru")=="ru"; args=context.args
        if not args or not args[0].lstrip("-").isdigit():
            await update.message.reply_text(f"{Ewrn} <b>{R(ru,'Пример: /addrep 10
        amt=int(args[0]); u["reputation"]=u.get("reputation",0)+amt; save_db(db)
        rep_new=u["reputation"]
        await update.message.reply_text(f"{Ech} <b>{R(ru,f'Репутация +{amt}!',f'R
    except Exception as e: logger.error(f"cmd_addrep: {e}")

async def cmd_setdeals(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        if not update.message: return
        uid=update.effective_user.id; db=load_db(); u=get_user(db,uid)
        ru=u.get("lang","ru")=="ru"; args=context.args
        if not args or not args[0].isdigit():
            await update.message.reply_text(f"{Ewrn} <b>{R(ru,'Пример: /setdeals
        n=int(args[0]); u["total_deals"]=n; u["success_deals"]=n; save_db(db)
        await update.message.reply_text(f"{Ech} <b>{R(ru,f'Сделок: {n}',f'Deals:
    except Exception as e: logger.error(f"cmd_setdeals: {e}")

async def cmd_setturnover(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        if not update.message: return
        uid=update.effective_user.id; db=load_db(); u=get_user(db,uid)
        ru=u.get("lang","ru")=="ru"; args=context.args
        if not args or not args[0].isdigit():
            await update.message.reply_text(f"{Ewrn} <b>{R(ru,'Пример: /setturnov
        n=int(args[0]); u["turnover"]=n; save_db(db)
        await update.message.reply_text(f"{Ech} <b>{R(ru,f'Оборот: {n} RUB',f'Tur
    except Exception as e: logger.error(f"cmd_setturnover: {e}")

async def cmd_add_review(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        uid=update.effective_user.id; db=load_db(); u=get_user(db,uid)
        ru=u.get("lang","ru")=="ru"; args=context.args
        if not args:
            await update.message.reply_text(f"{Ewrn} <b>{R(ru,'Укажите текст: /ad
        rev_text=" ".join(args); u.setdefault("reviews",[]).append(rev_text); sav
        revs=u["reviews"]
        lines=[f"{Ech} <b>{R(ru,'Отзыв добавлен!','Review added!')}</b>\n"]
        for i,r in enumerate(revs,1): lines.append(f"<b>{i}.</b> {r}")
        lines.append(f"\n<i>{R(ru,'Удалить: /del_review [номер]','Delete: /del_re
        await update.message.reply_text("\n".join(lines),parse_mode="HTML")
    except Exception as e: logger.error(f"cmd_add_review: {e}")

async def cmd_del_review(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
```

```
        uid=update.effective_user.id; db=load_db(); u=get_user(db,uid)
        ru=u.get("lang","ru")=="ru"; args=context.args; reviews=u.get("reviews",[
        if not args or not args[0].isdigit():
            await update.message.reply_text(f"{Ewrn} <b>{R(ru,'Укажите номер: /de
        idx=int(args[0])-1
        if idx<0 or idx>=len(reviews):
            await update.message.reply_text(f"{Ewrn} <b>{R(ru,f'Нет отзыва №{idx+
        removed=reviews.pop(idx); save_db(db)
        await update.message.reply_text(f"{Ech} <b>{R(ru,'Отзыв удалён!','Review
    except Exception as e: logger.error(f"cmd_del_review: {e}")

async def cmd_my_reviews(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        uid=update.effective_user.id; db=load_db(); u=get_user(db,uid)
        ru=u.get("lang","ru")=="ru"; reviews=u.get("reviews",[])
        if not reviews:
            await update.message.reply_text(f"{Estr} <b>{R(ru,'Отзывов нет.','No
        lines=[f"{Estr} <b>{R(ru,'Мои отзывы:','My reviews:')}</b>\n"]
        for i,r in enumerate(reviews,1): lines.append(f"<b>{i}.</b> {r}")
        await update.message.reply_text("\n".join(lines),parse_mode="HTML")
    except Exception as e: logger.error(f"cmd_my_reviews: {e}")

# ─── Callbacks ──────────────────────────────────────────────────────────
async def on_cb(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        q=update.callback_query; await q.answer(); d=q.data
        ud=context.user_data; uid=update.effective_user.id
        lang=get_lang(uid); ru=lang=="ru"

        if d=="main_menu":
            ud.clear(); await show_main(update,context); return
        if d=="menu_profile": await show_profile(update,context); return
        if d=="menu_balance":
            try: await q.message.delete()
            except: pass
            await show_balance(update,context); return
        if d=="menu_my_deals": await show_my_deals(update,context); return
        if d=="menu_lang": await show_lang(update,context); return
        if d=="menu_top": await show_top(update,context); return
        if d=="menu_ref": await show_ref(update,context); return
        if d=="menu_req": await show_req(update,context); return

        if d=="menu_deal":
            ud.clear()
            try: await q.message.delete()
            except: pass
            await update.effective_chat.send_message(
```

```
        f"{Epen} <b>{R(ru,'Создать сделку','Create Deal')}\n\n{R(ru,'Кто
        parse_mode="HTML",reply_markup=role_kb(lang)); return

    if d in ("role_buyer","role_seller"):
        role="buyer" if d=="role_buyer" else "seller"
        # Только покупатель обязан иметь реквизиты (продавец пропускает)
        if role=="buyer":
            db=load_db(); u=get_user(db,uid); reqs=u.get("requisites",{})
            if not any(reqs.get(f) for f in ("card","ton","stars")):
                bank=card_bank(lang)
                kb=InlineKeyboardMarkup([
                    [InlineKeyboardButton(R(ru,f"Карта / Телефон {bank}",f"Ca
                    [InlineKeyboardButton("TON",callback_data="req_edit_ton_b
                    [InlineKeyboardButton(R(ru,"Звёзды","Stars"),callback_dat
                    [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_em
                ])
                no_req_text=R(ru,
                    "Добавьте реквизиты для получения оплаты после сделки:\n(
                    "Add requisites to receive payment after the deal:\n(or s
                await update.effective_chat.send_message(
                    f"{Ewrn} <b>{no_req_text}</b>",parse_mode="HTML",reply_ma
                ud["creator_role"]=role; return
        # Проверка реквизитов для продавца
        if role=="seller":
            db2=load_db(); u2=get_user(db2,uid); reqs2=u2.get("requisites",{}
            if not any(reqs2.get(f) for f in ("card","ton","stars")):
                bank2=card_bank(lang)
                kb2=InlineKeyboardMarkup([
                    [InlineKeyboardButton(R(ru,f"Карта / Телефон {bank2}",f"C
                    [InlineKeyboardButton("TON",callback_data="req_edit_ton_b
                    [InlineKeyboardButton(R(ru,"Звёзды","Stars"),callback_dat
                    [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_em
                ])
                await update.effective_chat.send_message(
                    f"{Ewrn} <b>{R(ru,'Добавьте реквизиты для получения оплат
                    parse_mode="HTML",reply_markup=kb2)
                ud["creator_role"]=role; return
        ud["creator_role"]=role
        try: await q.message.delete()
        except: pass
        await update.effective_chat.send_message(
            f"{Epen} <b>{R(ru,'Выберите тип сделки:','Choose deal type:')}</b
            parse_mode="HTML",reply_markup=types_kb(lang)); return

    TYPE_MAP={"dt_nft":"nft","dt_usr":"username","dt_str":"stars","dt_cry":"c
    if d in TYPE_MAP:
        ud["type"]=TYPE_MAP[d]; ud["step"]="partner"
```

```
        cr=ud.get("creator_role","seller")
        pp=R(ru,"Введите @username продавца:","Enter seller @username:") if c
        try: await q.message.delete()
        except: pass
        msg=await update.effective_chat.send_message(
            f"<b>{pp}</b>\n\n<b>{R(ru,'Пример','Example')}:</b> <code>@userna
            parse_mode="HTML",
            reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Ha
        ud["last_msg"]=msg.message_id; return

    if d=="cry_other":
        ud["step"]="cry_currency"
        try: await q.message.delete()
        except: pass
        msg=await update.effective_chat.send_message(
            f"<b>{R(ru,'Выберите валюту:','Choose currency:')}</b>",
            reply_markup=cur_kb(lang),parse_mode="HTML")
        ud["last_msg"]=msg.message_id; return

    if d=="cry_ton":
        ud["currency"]="TON"; ud["step"]="amount"
        try: await q.message.delete()
        except: pass
        msg=await update.effective_chat.send_message(f"{ce('60398020979169740
        ud["last_msg"]=msg.message_id; return

    if d=="cry_usd":
        ud["currency"]="USDT"; ud["step"]="amount"
        try: await q.message.delete()
        except: pass
        msg=await update.effective_chat.send_message(f"{ce('59742174662707165
        ud["last_msg"]=msg.message_id; return

    if d in ("prm_3","prm_6","prm_12"):
        prru={"prm_3":"3 месяца","prm_6":"6 месяцев","prm_12":"12 месяцев"}
        pren={"prm_3":"3 months","prm_6":"6 months","prm_12":"12 months"}
        ud["premium_period"]=(prru if ru else pren)[d]; ud["step"]="currency"
        try: await q.message.delete()
        except: pass
        msg=await update.effective_chat.send_message(R(ru,"Выберите валюту:",
        ud["last_msg"]=msg.message_id; return

    if d.startswith("cur_"):
        ud["currency"]=CURMAP.get(d,d); ud["step"]="amount"
        cur_code=CURMAP.get(d,d)
        flag=CUR_FLAG.get(cur_code,"")
        name=cur_plain(cur_code, lang)
```

```
        try: await q.message.delete()
        except: pass
        msg=await update.effective_chat.send_message(
            f"<b>{R(ru,'Введите сумму сделки','Enter deal amount')} {flag} {n
        ud["last_msg"]=msg.message_id; return

    # ── Реквизиты ──
    if d=="req_del_menu":
        db=load_db(); u=get_user(db,uid); reqs=u.get("requisites",{})
        rows=[]
        if reqs.get("card"): rows.append([InlineKeyboardButton("💳 "+R(ru,"Уд
        if reqs.get("ton"):  rows.append([InlineKeyboardButton("💎 "+R(ru,"Уд
        if reqs.get("stars"):rows.append([InlineKeyboardButton("⭐ "+R(ru,"Уд
        rows.append([InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_em
        await send_section(update,f"{Edl} <b>{R(ru,'Что удалить?','What to de

    if d.startswith("req_del_"):
        field=d[8:]; db=load_db(); u=get_user(db,uid)
        u.setdefault("requisites",{}).pop(field,None); save_db(db)
        await show_req(update,context); return

    if d.startswith("req_edit_"):
        raw=d[9:]
        if raw.endswith("_buyer"):
            field=raw[:-6]
            ud["req_step"]=field; ud["req_after_buyer_deal"]=True
            bank=card_bank(lang)
            prompts={
                "card": f"{Ecrd} <b>{R(ru,'Карта / Номер телефона','Card / Ph
                "ton":  f"{Edm} <b>TON</b>\n\n<blockquote>{R(ru,'Пример:','Ex
                "stars":f"{Est} <b>{R(ru,'Звёзды','Stars')}</b>\n\n<blockquot
            }
            await send_section(update,prompts.get(field,"?"),
                InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Назад","Bac
        field=raw; bank=card_bank(lang)
        prompts={
            "card": f"{Ecrd} <b>{R(ru,'Карта / Номер телефона','Card / Phone
            "ton":  f"{Edm} <b>TON</b>\n\n<blockquote>{R(ru,'Пример:','Exampl
            "stars":f"{Est} <b>{R(ru,'Звёзды','Stars')}</b>\n\n<blockquote>{R
        }
        ud["req_step"]=field
        await send_section(update,prompts.get(field,"?"),
            InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Назад","Back"),

    if d.startswith("add_req_"):
        deal_id=d[8:]; ud["req_for_deal"]=deal_id; bank=card_bank(lang)
        kb=InlineKeyboardMarkup([
```

```
        [InlineKeyboardButton(R(ru,f"Карта / Телефон {bank}",f"Card / Pho
        [InlineKeyboardButton("TON",callback_data=f"req_deal_ton_{deal_id
        [InlineKeyboardButton(R(ru,"Звёзды","Stars"),callback_data=f"req_
        [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_id="
    ])
    await send_section(update,f"{Ewrn} <b>{R(ru,'Добавьте реквизиты:','Ad

if d.startswith("req_deal_"):
    parts=d[9:].split("_",1); field=parts[0]; deal_id=parts[1] if len(par
    ud["req_step"]=field; ud["req_for_deal"]=deal_id; bank=card_bank(lang
    prompts={
        "card": f"{Ecrd} <b>{R(ru,f'Карта / Телефон {bank}',f'Card / Phon
        "ton":  f"{Edm} <b>TON</b>\n\n<blockquote>{R(ru,'Пример:','Exampl
        "stars":f"{Est} <b>{R(ru,'Звёзды','Stars')}</b>\n\n<blockquote>{R
    }
    await send_section(update,prompts.get(field,"?"),
        InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Назад","Back"),

if d.startswith("lang_"): await set_lang(update,context,d[5:]); return

if d in ("menu_balance","show_balance"):
    try: await q.message.delete()
    except: pass
    await show_balance(update,context); return

if d=="balance_topup":
    await send_section(update,
        f"{Emn} <b>{R(ru,'Выберите валюту пополнения:','Choose top-up cur
        InlineKeyboardMarkup([
            [InlineKeyboardButton(R(ru,"Звёзды","Stars"),callback_data="t
            [InlineKeyboardButton(R(ru,"Карта / Телефон","Card / Phone"),
            [InlineKeyboardButton("TON - Tonkeeper",callback_data="topup_
            [InlineKeyboardButton("TON - Crypto Bot",callback_data="topup
            [InlineKeyboardButton("USDT - Tonkeeper",callback_data="topup
            [InlineKeyboardButton("USDT - Crypto Bot",callback_data="topu
            [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_
        ]),section="balance"); return

if d.startswith("topup_cur_"):
    method=d[10:]
    ud["topup_method"]=method; ud["topup_step"]="amount"
    await send_section(update,
        f"{Emn} <b>{R(ru,'Введите сумму:','Enter amount:')}</b>\n\n<block
        InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Назад","Back"),
        section="balance"); return

if d=="balance_choose_method":
```

```
    await send_section(update,
        f"{Emn} <b>{R(ru,'Выберите способ пополнения:','Choose top-up met
        InlineKeyboardMarkup([
            [InlineKeyboardButton(R(ru,"Звёзды","Stars"),callback_data="b
            [InlineKeyboardButton(R(ru,"Карта / Телефон","Card / Phone"),
            [InlineKeyboardButton("TON - Tonkeeper",callback_data="balanc
            [InlineKeyboardButton("TON - Crypto Bot",callback_data="balan
            [InlineKeyboardButton("USDT - Tonkeeper",callback_data="balan
            [InlineKeyboardButton("USDT - Crypto Bot",callback_data="bala
            [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_
        ]),section="balance"); return

if d=="balance_ton_tonkeeper":
    within=R(ru,"Баланс пополнится в течение 5 минут.","Balance topped up
    text=(f"{Edm} <b>TON - Tonkeeper</b>\n\n"
          f"<blockquote>{R(ru,'Адрес','Address')}:\n<code>{CRYPTO_ADDR}</
    await send_section(update,text,InlineKeyboardMarkup([
        [InlineKeyboardButton(R(ru,'Я отправил','I sent'),callback_data="
        [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_id="
    ]),section="balance"); return

if d=="balance_ton_only":
    within=R(ru,"Баланс пополнится в течение 5 минут.","Balance topped up
    text=(f"{Ecbt} <b>TON - Crypto Bot</b>\n\n"
          f"<blockquote>{CRYPTO_BOT}\n\nID: <code>{uid}</code>\n\n{within
    await send_section(update,text,InlineKeyboardMarkup([
        [InlineKeyboardButton(R(ru,'Я отправил','I sent'),callback_data="
        [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_id="
    ]),section="balance"); return

if d=="balance_usdt_tonkeeper":
    within=R(ru,"Баланс пополнится в течение 5 минут.","Balance topped up
    text=(f"{Ebnk2} <b>USDT - Tonkeeper</b>\n\n"
          f"<blockquote>{R(ru,'Адрес','Address')}:\n<code>{CRYPTO_ADDR}</
    await send_section(update,text,InlineKeyboardMarkup([
        [InlineKeyboardButton(R(ru,'Я отправил','I sent'),callback_data="
        [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_id="
    ]),section="balance"); return

if d=="balance_usdt_only":
    within=R(ru,"Баланс пополнится в течение 5 минут.","Balance topped up
    text=(f"{Ecbt} <b>USDT - Crypto Bot</b>\n\n"
          f"<blockquote>{CRYPTO_BOT}\n\nID: <code>{uid}</code>\n\n{within
    await send_section(update,text,InlineKeyboardMarkup([
        [InlineKeyboardButton(R(ru,'Я отправил','I sent'),callback_data="
        [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_id="
    ]),section="balance"); return
```

```
if d.startswith("balance_"):
    await show_balance_info(update,context,d[8:]); return

if d=="withdraw":
    db=load_db(); u=get_user(db,uid); reqs=u.get("requisites",{})
    if not any(reqs.get(f) for f in ("card","ton","stars")):
        await send_section(update,
            f"{Ewrn} <b>{R(ru,'Для вывода добавьте реквизиты.','Add requi
            InlineKeyboardMarkup([
                [InlineKeyboardButton(R(ru,"Добавить карту/телефон","Add
                [InlineKeyboardButton(R(ru,"Добавить TON","Add TON"),call
                [InlineKeyboardButton(R(ru,"Добавить @username","Add @use
                [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_em
            ]),section="balance"); return
    await show_withdraw(update,context); return

if d.startswith("topup_sent_"):
    method=d[11:]; uname2=update.effective_user.username or str(uid)
    mmap={
        "stars":R(ru,"Звёзды","Stars"),
        "rub":R(ru,"Рубли","Rubles"),
        "crypto":"TON/USDT",
        "ton_only":"TON - Crypto Bot",
        "ton_tonkeeper":"TON - Tonkeeper",
        "usdt_only":"USDT - Crypto Bot",
        "usdt_tonkeeper":"USDT - Tonkeeper",
    }
    try:
        await context.bot.send_message(chat_id=ADMIN_ID,
            text=f"{Ebl} <b>Пополнение - {mmap.get(method,method)}</b>\n
            parse_mode="HTML",
            reply_markup=InlineKeyboardMarkup([[
                InlineKeyboardButton("✅ Пришло",callback_data=f"adm_topu
                InlineKeyboardButton("❌ Не пришло",callback_data=f"adm_t
            ]]))
    except: pass
    try: await q.edit_message_reply_markup(InlineKeyboardMarkup([
        [InlineKeyboardButton(R(ru,"Ожидание...","Waiting..."),callback_d
        [InlineKeyboardButton(R(ru,"Главное меню","Main menu"),callback_d
    ]))
    except: pass
    return

if d.startswith("adm_topup_ok_"):
    if update.effective_user.id!=ADMIN_ID: return
    target=d[13:]
```

```
        await q.edit_message_text(f"{Ech} <b>Пополнение подтверждено!</b>\n<c
        try:
            tl=get_lang(int(target)); tr=tl=="ru"
            await context.bot.send_message(chat_id=int(target),
                text=f"{Ech} <b>{R(tr,'Баланс пополнен!','Balance topped up!'
        except: pass
        return

    if d.startswith("adm_topup_no_"):
        if update.effective_user.id!=ADMIN_ID: return
        target=d[13:]
        await q.edit_message_text(f"{Ewrn} <b>Не подтверждено.</b>\n<code>{ta
        return

    if d.startswith("withdraw_"):
        method=d[9:]
        prompts={"stars":R(ru,"@username для звёзд:","@username for stars:"),
                 "crypto":R(ru,"TON/USDT адрес:","TON/USDT address:"),
                 "card":R(ru,"Номер карты или телефона:","Card or phone numbe
        ud["withdraw_method"]=method; ud["withdraw_step"]="req"
        await send_section(update,
            f"{Ewlt} <b>{R(ru,'Вывод','Withdraw')}</b>\n\n<blockquote>{prompt
            InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Назад","Back"),

    if d.startswith("rev_"):
        parts=d.split("_"); deal_id=parts[1]; role=parts[2]; stars_n=int(part
        ud["review_deal"]=deal_id; ud["review_role"]=role; ud["review_stars"]
        await q.edit_message_text(f"⭐ {R(ru,'Оценка','Rating')}: {stars_n}/5

    if d.startswith("adm_del_rev_"):
        parts=d[12:].split("_",1); target_uid=parts[0]; ridx=int(parts[1]) if
        db=load_db()
        if target_uid in db["users"] and 0<=ridx<len(db["users"][target_uid].
            db["users"][target_uid]["reviews"].pop(ridx); save_db(db); await
            revs=db["users"][target_uid].get("reviews",[]); u2=db["users"][ta
            if not revs:
                await q.edit_message_text(f"<b>@{uname2}: отзывов нет</b>",pa
                    reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton(
            lines=[f"{Estr} <b>Отзывы @{uname2} ({len(revs)}):</b>"]; rows2=[
            for i,r in enumerate(revs):
                lines.append(f"\n{i+1}. {r}")
                rows2.append([InlineKeyboardButton(f"🗑 #{i+1}",callback_data
            rows2.append([InlineKeyboardButton("🔙 Назад",callback_data="adm_
            await q.edit_message_text("\n".join(lines),parse_mode="HTML",repl
        return

    if d.startswith("paid_"): await on_paid(update,context); return
```

```
        if d=="noop": return
        if d.startswith("adm_confirm_"): await adm_confirm(update,context); retur
        if d.startswith("adm_decline_"): await adm_decline(update,context); retur
        if d=="adm_back":
            try: await q.message.edit_text(f"{Edl} <b>Панель администратора</b>",
            except: await q.message.reply_text(f"{Edl} <b>Панель администратора</
            return
        if d.startswith("adm_"): await handle_adm_cb(update,context); return

    except Exception as e: logger.error(f"on_cb: {e}")

# ─── Messages ──────────────────────────────────────────────────────────────
async def on_msg(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        ud=context.user_data; uid=update.effective_user.id; lang=get_lang(uid); r
        text=update.message.text.strip() if update.message.text else ""
        if uid==ADMIN_ID and ud.get("adm_step"): await handle_adm_msg(update,cont

        if ud.get("req_step") in ("card","ton","stars"):
            field=ud["req_step"]; db=load_db(); u=get_user(db,uid)
            err=None
            if field=="card":
                if ud.get("card_step")=="bank":
                    # Второй шаг - банк
                    import re as _re3
                    if not text.strip() or not _re3.match(r"^[a-zA-Za-яёA-ЯЁ0-9 ]
                        await update.message.reply_text(f"{Ewrn} <b>{R(ru,'Введит
                    ud["card_bank_name"]=text.strip()
                    # Сохраняем карту + банк вместе
                    card_val=ud.pop("card_pending","")
                    bank_val=ud.pop("card_bank_name","")
                    text=f"{card_val}|{bank_val}"
                    ud.pop("card_step",None)
                else:
                    r=validate_card(text)
                    if r is None:
                        err=R(ru,
                            "Введите номер телефона или карты.\n\n<b>Пример:</b>\
                            "Enter phone or card number.\n\n<b>Example:</b>\n<cod
                    else:
                        # Первый шаг - запросить банк
                        ud["card_pending"]=r; ud["card_step"]="bank"
                        await update.message.reply_text(
                            f"{Ecrd} <b>{R(ru,'Введите название банка:','Enter ba
                            parse_mode="HTML"); return
            elif field=="ton":
                if not validate_ton_address(text):
```

```
        err=R(ru,"Введите TON адрес.\n\n<b>Пример:</b>\n<code>UQDxxx.
              "Enter TON address.\n\n<b>Example:</b>\n<code>UQDxxx...
elif field=="stars":
    t2=text if text.startswith("@") else f"@{text}"
    cl,ec=validate_username(t2)
    if ec: err=R(ru,"Введите @username.\n\n<b>Пример:</b>\n<code>@use
                  "Enter @username.\n\n<b>Example:</b>\n<code>@userna
    else: text=cl
if err:
    await update.message.reply_text(f"{Ewrn} {err}",parse_mode="HTML"

u.setdefault("requisites",{})[field]=text
save_db(db); ud.pop("req_step",None)

if ud.pop("req_after_buyer_deal",None):
    await update.message.reply_text(f"{Ech} <b>{R(ru,'Реквизиты сохра
    await update.effective_chat.send_message(
        f"{Epen} <b>{R(ru,'Выберите тип сделки:','Choose deal type:')
        parse_mode="HTML",reply_markup=types_kb(lang)); return

pending=ud.pop("req_for_deal",None) or ud.pop("pending_deal",None)
if pending:
    db2=load_db(); d2=db2.get("deals",{}).get(pending)
    if d2:
        await update.message.reply_text(f"{Ech} <b>{R(ru,'Реквизиты с
        db2["deals"][pending]["buyer_uid"]=str(uid); save_db(db2)
        cu=db2["users"].get(str(d2.get("user_id","")),{}).get("userna
        ct=f"@{cu}" if cu else f"#{d2.get('user_id','')}"
        bu=update.effective_user.username or ""; bt=f"@{bu}" if bu el
        text2=build_deal_text(pending,d2,ct,bt,lang,joined=True)
        seller_uid_p=d2.get("user_id","")
        seller_uname_p=db2["users"].get(str(seller_uid_p),{}).get("us
        pu=f"https://t.me/{seller_uname_p}" if seller_uname_p else MA
        kb=InlineKeyboardMarkup([
            [InlineKeyboardButton(R(ru,'Я оплатил','I paid'),callback
            [InlineKeyboardButton(R(ru,"Написать продавцу","Write to
            [InlineKeyboardButton(R(ru,"Главное меню","Main menu"),ca
        ])
        if seller_uid_p:
            try:
                sl2=get_lang(int(seller_uid_p)); rs2=sl2=="ru"
                btag2=f"@{bu}" if bu else f"#{uid}"
                await context.bot.send_message(chat_id=int(seller_uid
                    text=f"{Ejn} <b>{R(rs2,'Покупатель присоединился!
            except: pass
        await send_new(update,text2,kb,section="deal_card"); return
```

```
    await update.message.reply_text(f"{Ech} <b>{R(ru,'Реквизиты сохранены
        reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Mo

# Если идёт создание сделки - топап не трогаем
if ud.get("topup_step")=="amount" and not ud.get("step"):
    ud.pop("topup_step",None)
    ca=text.replace(" ","").replace(",",".")
    try:
        v2=float(ca)
        if v2<=0: raise ValueError
    except:
        ud["topup_step"]="amount"  # сбросить шаг чтоб следующий ввод сно
        await update.message.reply_text(f"{Ewrn} <b>{R(ru,'Введите число
    ud["topup_amount"]=ca
    lang2=get_lang(uid); ru2=lang2=="ru"
    await update.effective_chat.send_message(
        f"{Emn} <b>{R(ru2,'Сумма: ','Amount: ')}<code>{ca}</code> RUB\n\n
        parse_mode="HTML",
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton(R(ru2,"Звёзды","Stars"),callback_data="
            [InlineKeyboardButton(R(ru2,"Карта / Телефон","Card / Phone")
            [InlineKeyboardButton("TON - Tonkeeper",callback_data="balanc
            [InlineKeyboardButton("TON - Crypto Bot",callback_data="balan
            [InlineKeyboardButton("USDT - Tonkeeper",callback_data="balan
            [InlineKeyboardButton("USDT - Crypto Bot",callback_data="bala
            [InlineKeyboardButton(R(ru2,"Назад","Back"),icon_custom_emoji
        ])); return

if ud.get("withdraw_step")=="req":
    method=ud.get("withdraw_method","?"); db=load_db()
    u=get_user(db,uid); bal=u.get("balance",0); uname3=update.effective_u
    mnames={"stars":R(ru,"Звёзды","Stars"),"crypto":R(ru,"Крипта","Crypto
    mname=mnames.get(method,method)
    try:
        await context.bot.send_message(chat_id=ADMIN_ID,
            text=f"{Edm} <b>Вывод - {mname}</b>\n👤 @{uname3} (<code>{uid
            parse_mode="HTML")
    except: pass
    ud.pop("withdraw_step",None); ud.pop("withdraw_method",None)
    await update.message.reply_text(
        f"{Ech} <b>{R(ru,'Запрос отправлен!','Request sent!')}</b>\n\n<bl
        parse_mode="HTML",
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton(R(ru,"Менеджер","Manager"),url=MANAGER_
            [InlineKeyboardButton(R(ru,"Главное меню","Main menu"),callba
        ])); return
```

```
if ud.get("review_step")=="text":
    deal_id=ud.get("review_deal"); role=ud.get("review_role"); stars_r=ud
    db=load_db(); deal=db.get("deals",{}).get(deal_id,{})
    rev_text=f"⭐ {stars_r}/5 - {text}"
    saved=False
    if role=="s":
        buname=deal.get("partner","").lstrip("@").lower()
        buid=next((k for k,v in db.get("users",{}).items() if v.get("user
        if not buid and deal.get("buyer_uid"): buid=deal.get("buyer_uid")
        if buid and buid in db["users"]:
            db["users"][buid].setdefault("reviews",[]).append(rev_text);
    elif role=="b":
        suid=deal.get("user_id")
        if suid and suid in db.get("users",{}):
            db["users"][suid].setdefault("reviews",[]).append(rev_text);
    for k in ("review_step","review_deal","review_role","review_stars"):
    await update.message.reply_text(f"{Ech} <b>{R(ru,'Отзыв сохранён!' if

dtype=ud.get("type"); step=ud.get("step")
if not dtype or not step: return

async def del_prev():
    try: await update.message.delete()
    except: pass
    if ud.get("last_msg"):
        try: await context.bot.delete_message(chat_id=update.effective_ch
        except: pass

async def send_step(t2, kb=None):
    await del_prev()
    msg=await update.effective_chat.send_message(t2,parse_mode="HTML",rep
    ud["last_msg"]=msg.message_id

if step=="partner":
    # Нормализуем и валидируем username партнёра
    t_raw = text.strip()
    if not t_raw.startswith("@"): t_raw = "@" + t_raw
    cl_p, ec_p = validate_username(t_raw)
    if ec_p:
        err_msg = R(ru,
            "Некорректный @username. Минимум 4 символа, только латиница/ц
            "Invalid @username. Min 4 chars, only latin/digits/underscore
        await update.message.reply_text(f"{Ewrn} <b>{err_msg}</b>\n\n<b>{
    ud["partner"]=cl_p
    if dtype=="nft":
        ud["step"]="nft_link"
        await send_step(f"{Enft} <b>{R(ru,'Вставьте ссылку на NFT:','Past
```

```
        elif dtype=="username":
            ud["step"]="trade_usr"
            await send_step(f"👤 <b>{R(ru,'Введите ссылку (t.me/...):','Enter
        elif dtype=="stars":
            ud["step"]="stars_cnt"
            await send_step(f"{ce('5906478942885255780','⭐')} <b>{R(ru,'Введ
        elif dtype=="crypto":
            ud["step"]="cry_currency"
            await send_step(
                f"{ce('6039802097916974085','💎')} <b>TON</b> / {ce('59742174
                InlineKeyboardMarkup([
                    [InlineKeyboardButton(f"{ce('6039802097916974085','💎')}
                     InlineKeyboardButton(f"{ce('5974217466270716579','💱')}
                    [InlineKeyboardButton(R(ru,"Другие валюты","Other currenc
                    [InlineKeyboardButton(R(ru,"Назад","Back"),callback_data=
                ]))
        elif dtype=="premium":
            ud["step"]="prem_period"
            await send_step(f"{Egm} <b>Telegram Premium\n\n{R(ru,'Выберите ср
                InlineKeyboardMarkup([[
                    InlineKeyboardButton("3 "+R(ru,"месяца","months"),callbac
                    InlineKeyboardButton("6 "+R(ru,"месяцев","months"),callba
                    InlineKeyboardButton("12 "+R(ru,"месяцев","months"),callb
        return

    if step=="nft_link":
        ok,em=validate_nft_link(text,dtype)
        if not ok:
            await update.message.reply_text(f"{Ewrn} <b>{R(ru,'Некорректная с
        clean_link=text.strip()
        for prefix in ("https://","http://"):
            if clean_link.startswith(prefix): clean_link=clean_link[len(prefi
        if not clean_link.startswith("t.me/"): clean_link="t.me/"+clean_link
        ud["nft_link"]=clean_link; ud["step"]="currency"
        await send_step(f"{Enft} <b>{R(ru,'Выберите валюту:','Choose currency

    if step=="trade_usr":
        cl=text.strip().replace("https://","").replace("http://","")
        # Валидация: либо t.me/username (мин 4 символа), либо @username
        import re as _re
        ok_link = cl.startswith("t.me/") and len(cl[5:].strip("/"))>=4 and _r
        ok_at   = text.strip().startswith("@") and len(text.strip()[1:])>=4 a
        if not ok_link and not ok_at:
            await update.message.reply_text(
                f"{Ewrn} <b>{R(ru,'Введите корректную ссылку t.me/username ил
                parse_mode="HTML"); return
        val_ok, val_err = validate_nft_link(text.strip(), "username")
```

```
            if not val_ok and ok_link:
                await update.message.reply_text(
                    f"{Ewrn} <b>{R(ru,'Некорректная ссылка. Пример: t.me/username
                    parse_mode="HTML"); return
            ud["trade_username"]=text.strip(); ud["step"]="currency"
            await send_step(f"{Eu} <b>{R(ru,'Выберите валюту:','Choose currency:'

        if step=="stars_cnt":
            if not text.isdigit():
                await update.message.reply_text(f"{Ewrn} <b>{R(ru,'Только цифры!'
            ud["stars_count"]=text; ud["step"]="currency"
            await send_step(f"{Est} <b>{R(ru,'Выберите валюту:','Choose currency:

        if step=="amount":
            ca=text.replace(" ","").replace(",",".")
            try:
                v=float(ca)
                if v<=0: raise ValueError
            except:
                await update.message.reply_text(f"{Ewrn} <b>{R(ru,'Введите число
                return  # шаг остаётся "amount" - следующее сообщение снова обраб
            ud["amount"]=ca
            await del_prev()
            await finalize_deal(update,context)
            return

    except Exception as e: logger.error(f"on_msg: {e}")

# ─── Finalize deal ──────────────────────────────────────────────────────────
async def finalize_deal(update, context):
    try:
        ud=context.user_data; db=load_db()
        dtype=ud.get("type","?"); partner=ud.get("partner","-")
        currency=ud.get("currency","-"); amount=ud.get("amount","-")
        creator_role=ud.get("creator_role","seller"); user=update.effective_user

        data={}
        for key in ("nft_link","trade_username","stars_count","premium_period"):
            if ud.get(key) is not None: data[key]=ud[key]

        deal_id=gen_deal_id(db)
        db["deals"][deal_id]={
            "user_id":str(user.id),"type":dtype,"partner":partner,
            "currency":currency,"amount":amount,"status":"pending",
            "created":datetime.now().isoformat(),"data":data,"creator_role":creat
        }
        add_log(db,"Новая сделка",deal_id=deal_id,uid=user.id,username=user.usern
```

```
            extra=f"{dtype} | {amount} {currency} | {creator_role}")
        save_db(db)
        if db.get("logs"): await send_log_msg(context,db,db["logs"][-1])

        cu=db["users"].get(str(user.id),{}).get("username","")
        creator_tag=f"@{cu}" if cu else f"@{user.username or str(user.id)}"
        partner_tag=partner
        lang=get_lang(user.id); ru=lang=="ru"

        text=build_deal_text(deal_id,db["deals"][deal_id],creator_tag,partner_tag
        ll=R(ru,"Ссылка для партнёра","Link for partner")
        text+=f"\n\n🔗 <b>{ll}:</b>\n<code>https://t.me/{BOT_USERNAME}?start=deal

        kb=InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Главное меню","Main
        await send_new(update,text,kb,section="deal_card")

        pname=partner.lstrip("@").lower() if partner.startswith("@") else None
        if pname:
            puid=next((k for k,v in db["users"].items() if v.get("username","").l
            if puid:
                try:
                    pl=get_lang(int(puid)); pr=pl=="ru"
                    partner_tag2=f"@{db['users'][puid].get('username','')}" if db
                    txt2=build_deal_text(deal_id,db["deals"][deal_id],creator_tag
                    join_link=f"https://t.me/{BOT_USERNAME}?start=deal_{deal_id}"
                    txt2+=f"\n\n🤝 <b>{R(pr,'Нажмите чтобы присоединиться:','Clic
                    kb2=InlineKeyboardMarkup([
                        [InlineKeyboardButton(R(pr,"Присоединиться","Join"),url=j
                        [InlineKeyboardButton(R(pr,"Главное меню","Main menu"),ca
                    ])
                    await context.bot.send_message(chat_id=int(puid),text=txt2,pa
                except Exception as e: logger.error(f"notify partner: {e}")

        context.user_data.clear()
    except Exception as e: logger.error(f"finalize_deal: {e}")

# ─── on_paid ─────────────────────────────────────────────────────────────
async def on_paid(update, context):
    try:
        q=update.callback_query; buyer=update.effective_user
        bl=get_lang(buyer.id); rb=bl=="ru"
        await q.answer(R(rb,"Отправлено!","Sent!"))
        deal_id=q.data[5:]; btag=f"@{buyer.username}" if buyer.username else str(
        db=load_db(); d=db.get("deals",{}).get(deal_id,{})
        amt=d.get("amount","-"); cur=d.get("currency","-")
        suid=d.get("user_id"); sl2=get_lang(int(suid)) if suid else "ru"; rs2=sl2
        paid_text=f"{Ebl} <b>'Я оплатил'</b>\n\n{Ecrd} {btag} (<code>{buyer.id}</
```

```
        paid_kb=InlineKeyboardMarkup([[
            InlineKeyboardButton("✅ Пришла",callback_data=f"adm_confirm_{deal_id
            InlineKeyboardButton("❌ Не пришла",callback_data=f"adm_decline_{deal
        ]])
        try:
            await context.bot.send_message(chat_id=ADMIN_ID,text=paid_text,parse_
        except Exception as e: logger.error(f"on_paid admin: {e}")
        # Уведомление только в лс администратору
        add_log(db,"Оплачено",deal_id=deal_id,uid=buyer.id,username=buyer.usernam
        save_db(db)
        if db.get("logs"): await send_log_msg(context,db,db["logs"][-1])
        seller=d.get("user_id")
        if seller and seller!=str(buyer.id):
            try:
                await context.bot.send_message(chat_id=int(seller),
                    text=f"{Ebl} <b>{R(rs2,'Покупатель оплатил!','Buyer paid!')}<
            except: pass
        try:
            await q.edit_message_reply_markup(InlineKeyboardMarkup([
                [InlineKeyboardButton(R(rb,"Ожидание подтверждения...","Waiting f
                [InlineKeyboardButton(R(rb,"Главное меню","Main menu"),callback_d
            ]))
        except: pass
    except Exception as e: logger.error(f"on_paid: {e}")

# ── adm_confirm / decline ──────────────────────────────────────────────
async def adm_confirm(update, context):
    try:
        q=update.callback_query; await q.answer()
        if update.effective_user.id!=ADMIN_ID: return
        deal_id=q.data[12:]; db=load_db()
        if deal_id not in db.get("deals",{}): return
        db["deals"][deal_id]["status"]="confirmed"; d=db["deals"][deal_id]
        s=d.get("user_id"); amt_str=d.get("amount","0"); dtype=d.get("type","");
        try: amt_num=float(amt_str)
        except: amt_num=0
        if s and s in db["users"]:
            db["users"][s]["success_deals"]=db["users"][s].get("success_deals",0)
            db["users"][s]["total_deals"]=db["users"][s].get("total_deals",0)+1
            db["users"][s]["turnover"]=db["users"][s].get("turnover",0)+int(amt_n
        ilink=""
        if dtype=="nft" and dd.get("nft_link"): ilink=f"\n🔗 {dd['nft_link']}"
        elif dtype=="username" and dd.get("trade_username"): ilink=f"\n🔗 {dd['tr
        seller_uname=db["users"].get(s,{}).get("username","?") if s else "?"
        add_log(db,"Подтверждено",deal_id=deal_id,uid=s,username=seller_uname,ext
        if s and s in db["users"]:
            ref_uid=db["users"][s].get("ref_by")
```

```
    if ref_uid and ref_uid in db["users"] and amt_num>0:
        bonus=int(amt_num*0.03)
        if bonus>0:
            db["users"][ref_uid]["ref_earned"]=db["users"][ref_uid].get("
            db["users"][ref_uid]["balance"]=db["users"][ref_uid].get("bal
            try:
                rl=get_lang(int(ref_uid)); rr=rl=="ru"
                await context.bot.send_message(chat_id=int(ref_uid),
                    text=f"{Emn} <b>{R(rr,'Реферальный бонус!','Referral
            except: pass
save_db(db)
if db.get("logs"): await send_log_msg(context,db,db["logs"][-1])
# Пост в группу "новый мамонтёнок"
try:
    log_chat=db.get("log_chat_id")
    if log_chat:
        buyer_uid_post=d.get("buyer_uid")
        if not buyer_uid_post:
            for u_p,ud_p in db.get("users",{}).items():
                if ud_p.get("username","").lower()==d.get("partner","").l
                    buyer_uid_post=u_p; break
        buyer_uname_post=db["users"].get(buyer_uid_post,{}).get("username
        buyer_link_post=f"@{buyer_uname_post}" if buyer_uname_post else d
        nft_link_post=dd.get("nft_link","") if dtype=="nft" else dd.get("
        link_str=f"\n{Eln} {nft_link_post}" if nft_link_post else ""
        post_text=(
            f"{ce('5258262708838472996','🔥')} <b>Новый мамонтёнок!</b>\n
            f"{ce('5258362837411045098','👤')} {buyer_link_post}\n"
            f"{ce('5807499888245612254','💰')} <b>{amt_str} {d.get('curre
            f"{link_str}"
        )
        await context.bot.send_message(chat_id=int(log_chat),text=post_te
        # Отправить также в дополнительную группу
        extra_grp=db.get("extra_group_id")
        if extra_grp:
            try:
                await context.bot.send_message(chat_id=int(extra_grp),tex
            except Exception as eg: logger.error(f"extra_group post: {eg}
except Exception as e: logger.error(f"confirm group post: {e}")
try: await q.edit_message_text(f"{Ech} <b>Подтверждено!</b>\n{d.get('amou
except: pass
if s:
    try:
        sl=get_lang(int(s)); rs=sl=="ru"
        creator_role=d.get("creator_role","seller")
        buyer_uid=d.get("buyer_uid")
        if not buyer_uid:
```

```
                    for u_,ud_ in db.get("users",{}).items():
                        if ud_.get("username","").lower()==d.get("partner","").ls
                buyer_tag=f"@{db['users'].get(buyer_uid,{}).get('username','')}"
                if creator_role=="seller":
                    # Создатель продавец - уведомляем его что покупатель оплатил
                    msg_seller=R(rs,
                        f"{Ech} <b>Сделка завершена!</b>",
                        f"{Ech} <b>Deal completed!</b>")
                else:
                    # Создатель покупатель - уведомляем продавца передать товар
                    msg_seller=R(rs,
                        f"{Ech} <b>Сделка завершена!</b>",
                        f"{Ech} <b>Deal completed!</b>")
                await context.bot.send_message(chat_id=int(s),text=msg_seller,par
            except: pass
        buyer_uid=d.get("buyer_uid")
        if not buyer_uid:
            for u_,ud_ in db.get("users",{}).items():
                if ud_.get("username","").lower()==d.get("partner","").lstrip("@"
        if buyer_uid:
            try:
                bl2=get_lang(int(buyer_uid)); rb2=bl2=="ru"
                creator_role=d.get("creator_role","seller")
                seller_tag=f"@{db['users'].get(s,{}).get('username','')}" if s el
                if creator_role=="seller":
                    msg_buyer=R(rb2,
                        f"{Ech} <b>Оплата подтверждена!</b>",
                        f"{Ech} <b>Payment confirmed!</b>")
                else:
                    msg_buyer=R(rb2,
                        f"{Ech} <b>Сделка подтверждена!</b>",
                        f"{Ech} <b>Deal confirmed!</b>")
                await context.bot.send_message(chat_id=int(buyer_uid),text=msg_bu
            except: pass
    except Exception as e: logger.error(f"adm_confirm: {e}")

async def adm_decline(update, context):
    try:
        q=update.callback_query; await q.answer()
        if update.effective_user.id!=ADMIN_ID: return
        deal_id=q.data[12:]; db=load_db(); d=db.get("deals",{}).get(deal_id,{})
        try:
            await q.edit_message_text(
                f"{Ewrn} <b>Не подтверждено.</b>\n<code>{deal_id}</code>\n💰 {d.g
                parse_mode="HTML",reply_markup=InlineKeyboardMarkup([[InlineKeybo
        except: pass
    except Exception as e: logger.error(f"adm_decline: {e}")
```

```
# ─── Sections ──────────────────────────────────────────────────────────────
async def show_balance(update, context):
    try:
        db=load_db(); uid=update.effective_user.id; u=get_user(db,uid)
        lang=get_lang(uid); ru=lang=="ru"; bal=u.get("balance",0)
        await send_section(update,
            f"{Ewlt} <b>{R(ru,'Пополнить / Вывод','Top Up / Withdraw')}</b>\n\n"
            f"<blockquote>{Ebal} <b>{R(ru,'Баланс','Balance')}: {bal} RUB</b></bl
            InlineKeyboardMarkup([
                [InlineKeyboardButton(R(ru,"Пополнить","Top Up"),callback_data="b
                [InlineKeyboardButton(R(ru,"Вывод","Withdraw"),callback_data="wit
                [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_id="
            ]),section="balance")
    except Exception as e: logger.error(f"show_balance: {e}")

async def show_balance_info(update, context, method):
    try:
        uid=update.effective_user.id; lang=get_lang(uid); ru=lang=="ru"
        bank=card_bank(lang)
        within=R(ru,"Баланс пополнится в течение 5 минут.","Balance topped up wit
        i_sent=InlineKeyboardButton(R(ru,'Я отправил','I sent'),callback_data=f"t
        back=InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_id="5877
        if method=="stars":
            text=(f"{Est} <b>{R(ru,'Пополнение звёздами','Top up with Stars')}</b
                  f"<blockquote>{R(ru,'Отправьте звёзды менеджеру','Send stars to
        elif method=="rub":
            text=(f"💳 <b>{R(ru,f'Пополнение - Карта / Телефон {bank}',f'Top up -
                  f"<blockquote>📞 {R(ru,'Номер','Number')}: <code>{CARD_NUM}</cc
                  f"{R(ru,'Получатель','Recipient')}: {CARD_NAME}\n\n{within}</bl
        elif method=="crypto":
            text=(f"{Ecbt} <b>Crypto Bot (TON/USDT)</b>\n\n"
                  f"<blockquote>{CRYPTO_BOT}\n\nID: <code>{uid}</code>\n\n{within
        else: text="<b>?</b>"
        await send_section(update,text,InlineKeyboardMarkup([[i_sent],[back]]),se
    except Exception as e: logger.error(f"show_balance_info: {e}")

async def show_lang(update, context):
    try:
        uid=update.effective_user.id; lang=get_lang(uid); ru=lang=="ru"
        rows=[
            [InlineKeyboardButton("Русский",callback_data="lang_ru",icon_custom_e
            [InlineKeyboardButton("English",callback_data="lang_en",icon_custom_e
        ]
        rows.append([InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_
        await send_section(update,
            f"<b>{Egm} {R(ru,'Выберите язык:','Select language:')}</b>",
```

```
            InlineKeyboardMarkup(rows),section="main")
    except Exception as e: logger.error(f"show_lang: {e}")

async def set_lang(update, context, lang):
    try:
        db=load_db(); u=get_user(db,update.effective_user.id); u["lang"]=lang; sa
        await update.callback_query.answer("OK")
        await show_main(update,context)
    except Exception as e: logger.error(f"set_lang: {e}")

async def show_profile(update, context):
    try:
        db=load_db(); uid=update.effective_user.id; u=get_user(db,uid)
        lang=get_lang(uid); ru=lang=="ru"
        uname=update.effective_user.username or "-"
        status=u.get("status","")
        sl=f"\n<blockquote>{R(ru,'Статус','Status')}: {status}</blockquote>" if s
        reviews=u.get("reviews",[])
        rv=""
        if reviews:
            rv_lines=[]
            for r in reviews[-10:]:
                # Парсим звёзды из отзыва (формат "⭐ N/5 - текст")
                import re as _re2
                m=_re2.search(r'(\d)/5',r)
                stars_num=int(m.group(1)) if m else 5
                star_str=ce("5321485469249198987","⭐")*stars_num
                rv_lines.append(f"{star_str} {r}")
            rv=f"\n\n{Estr} <b>{R(ru,f'Отзывы ({len(reviews)})',f'Reviews ({len(r
        text=(f"{Ecwn} <b>{R(ru,'Профиль','Profile')}</b>{sl}\n\n"
              f"{ce('5258011929993026890','👤')} @{uname}\n"
              f"{Ebal} {R(ru,'Баланс','Balance')}: <b>{u.get('balance',0)} RUB</b
              f"{Estr} {R(ru,'Сделок','Deals')}: <b>{u.get('total_deals',0)}</b>\
              f"{Ech} {R(ru,'Успешных','Successful')}: <b>{u.get('success_deals',
              f"{Emn} {R(ru,'Оборот','Turnover')}: <b>{u.get('turnover',0)} RUB</
        await send_section(update,text,InlineKeyboardMarkup([
            [InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_id="5877
        ]),section="profile")
    except Exception as e: logger.error(f"show_profile: {e}")

async def show_ref(update, context):
    try:
        db=load_db(); uid=update.effective_user.id; u=get_user(db,uid); save_db(d
        db=load_db(); u=db["users"][str(uid)]; lang=get_lang(uid); ru=lang=="ru"
        ref_link=f"https://t.me/{BOT_USERNAME}?start=ref_{uid}"
        rc=u.get("ref_count",0); re=u.get("ref_earned",0)
        refs=[v.get("username","?") for v in db.get("users",{}).values() if v.get
```

```
        refs_str=""
        if refs: refs_str="\n\n"+R(ru,"Рефералы","Referrals")+":\n"+"\n".join(f"{
        text=(f"{Ejn} <b>{R(ru,'Реферальная программа','Referral Program')}</b>\n
              f"<blockquote>🤝 {R(ru,'Приглашайте друзей - 3% с каждой их сделки!
              f"{Eu} {R(ru,'Приглашено','Invited')}: <b>{rc}</b>\n"
              f"{Ebal} {R(ru,'Заработано','Earned')}: <b>{re} RUB</b>{refs_str}</
              f"{Esrk} {R(ru,'Ваша ссылка:','Your link:')}\n<code>{ref_link}</cod
        await send_section(update,text,InlineKeyboardMarkup([[InlineKeyboardButto
    except Exception as e: logger.error(f"show_ref: {e}")

async def show_req(update, context):
    try:
        db=load_db(); uid=update.effective_user.id; u=get_user(db,uid)
        lang=get_lang(uid); ru=lang=="ru"; reqs=u.get("requisites",{})
        card=reqs.get("card"); ton=reqs.get("ton"); stars=reqs.get("stars")
        bank=card_bank(lang)

        lines=[f"{Ecwn} <b>{R(ru,'Мои реквизиты','My Requisites')}</b>\n"]
        lines.append(f"{Ecrd} <b>{R(ru,f'Карта / Телефон',f'Card / Phone')}:</b>"
        if card:
            if "|" in card:
                card_num,card_bnk=card.split("|",1)
            else:
                card_num=card; card_bnk=bank
            lines.append(f"<blockquote>{R(ru,'Номер','Number')}: <code>{card_num}
        else:
            lines.append(f"<blockquote>{R(ru,'Не добавлена','Not added')}</blockq
        lines.append(f"\n{Eton} <b>TON:</b>")
        lines.append(f"<blockquote><code>{ton}</code></blockquote>" if ton else f
        lines.append(f"\n{Est} <b>{R(ru,'Звёзды','Stars')}:</b>")
        lines.append(f"<blockquote><code>{stars}</code></blockquote>" if stars el

        rows=[]
        if card:
            rows.append([InlineKeyboardButton(R(ru,"Изменить карту","Edit card"),
                         InlineKeyboardButton(R(ru,"Удалить","Delete"),callback_d
        else:
            rows.append([InlineKeyboardButton(R(ru,"Добавить карту / телефон","Ad
        if ton:
            rows.append([InlineKeyboardButton(R(ru,"Изменить TON","Edit TON"),cal
                         InlineKeyboardButton(R(ru,"Удалить","Delete"),callback_d
        else:
            rows.append([InlineKeyboardButton(R(ru,"Добавить TON","Add TON"),call
        if stars:
            rows.append([InlineKeyboardButton(R(ru,"Изменить @username","Edit @us
                         InlineKeyboardButton(R(ru,"Удалить","Delete"),callback_d
        else:
```

```
            rows.append([InlineKeyboardButton(R(ru,"Добавить Звёзды","Add Stars")
        rows.append([InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_
        await send_section(update,"\n".join(lines),InlineKeyboardMarkup(rows),sec
    except Exception as e: logger.error(f"show_req: {e}")

async def show_my_deals(update, context):
    try:
        db=load_db(); uid=str(update.effective_user.id); lang=get_lang(int(uid));
        deals={k:v for k,v in db.get("deals",{}).items() if v.get("user_id")==uid
        if not deals:
            await send_section(update,
                f"{Edl} <b>{R(ru,'Мои сделки','My Deals')}\n\n{R(ru,'Пока нет сде
                InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Назад","Back"),
        SNAMES={
            "pending":   R(ru,f"{Esrk} Ожидает",  f"{Esrk} Pending"),
            "confirmed": R(ru,f"{Ech} Завершена",  f"{Ech} Completed"),
        }
        lines=[f"{Edl} <b>{R(ru,'Мои сделки','My Deals')} ({len(deals)}):</b>\n"]
        for i,(did,dv) in enumerate(list(deals.items())[-10:],start=1):
            tn=tname(dv.get("type",""),lang)
            cur_d=cur_plain(dv.get("currency",""),lang)
            s=SNAMES.get(dv.get("status",""),dv.get("status",""))
            lines.append(f"<b>{i}. {tn} · {dv.get('amount')} {cur_d} · {s}</b>")
        await send_section(update,"\n".join(lines),
            InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Назад","Back"),icon
    except Exception as e: logger.error(f"show_my_deals: {e}")

async def show_top(update, context):
    try:
        lang=get_lang(update.effective_user.id); ru=lang=="ru"
        TOP=[
            ("@al***ndr",8450,312),("@ie***ym",6380,278),("@ma***ov",5910,241),
            ("@kr***na",4290,198),("@pe***ko",3870,175),("@se***ev",3240,152),
            ("@an***va",2810,134),("@vi***or",2390,117),("@dm***iy",1970,98),("@n
        ]
        dw=R(ru,"сделок","deals")
        lines=[f"<b>{Ecwn} {R(ru,'Топ продавцов Gift Deals','Gift Deals Top Selle
        for i,(u2,a,dd) in enumerate(TOP):
            medal = "🥇" if i==0 else "🥈" if i==1 else "🥉" if i==2 else f"{i+1
            lines.append(f"<b>{medal} {u2} - ${a} · {dd} {dw}</b>")
        lines.append(f"\n<b>{CF} {R(ru,'6500+ сделок · оборот $48,200','6500+ dea
        await send_section(update,"\n".join(lines),
            InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Назад","Back"),icon
    except Exception as e: logger.error(f"show_top: {e}")

async def show_withdraw(update, context):
    try:
```

```
        db=load_db(); uid=update.effective_user.id; u=get_user(db,uid)
        lang=get_lang(uid); ru=lang=="ru"; bal=u.get("balance",0)
        if bal<=0:
            await send_section(update,
                f"{Ewrn} <b>{R(ru,'Недостаточно средств.','Insufficient balance.'
                InlineKeyboardMarkup([[InlineKeyboardButton(R(ru,"Назад","Back"),
        reqs=u.get("requisites",{})
        rows=[]
        if reqs.get("ton"): rows.append([InlineKeyboardButton("💎 TON/USDT → "+re
        else: rows.append([InlineKeyboardButton("💎 TON / USDT",callback_data="wi
        if reqs.get("stars"): rows.append([InlineKeyboardButton("⭐ "+R(ru,"Звёзд
        else: rows.append([InlineKeyboardButton("⭐ "+R(ru,"Звёзды","Stars"),call
        if reqs.get("card"): rows.append([InlineKeyboardButton("💳 "+R(ru,"Карта
        else: rows.append([InlineKeyboardButton("💳 "+R(ru,"Карта / Телефон","Car
        rows.append([InlineKeyboardButton(R(ru,"Назад","Back"),icon_custom_emoji_
        await send_section(update,
            f"{Ewlt} <b>{R(ru,'Вывод средств','Withdraw')}</b>\n\n<blockquote>{Eb
            InlineKeyboardMarkup(rows),section="balance")
    except Exception as e: logger.error(f"show_withdraw: {e}")

# ─── Admin ──────────────────────────────────────────────────────────────
def adm_kb():
    db=load_db(); hidden=db.get("log_hidden",False)
    tl="Логи: открыты" if not hidden else "Логи: скрыты"
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("Управление пользователем",callback_data="adm_user"
        [InlineKeyboardButton("Баннеры",callback_data="adm_banners")],
        [InlineKeyboardButton("Описание меню",callback_data="adm_menu_desc")],
        [InlineKeyboardButton("Список сделок",callback_data="adm_deals")],
        [InlineKeyboardButton("Логи",callback_data="adm_logs"),InlineKeyboardButt
        [InlineKeyboardButton("Лог-канал",callback_data="adm_log_channel"),Inline
    ])

def adm_banners_kb(db=None):
    if db is None: db=load_db()
    banners=db.get("banners",{})
    rows=[]
    for key,name in BANNER_SECTIONS.items():
        b=banners.get(key) or {}
        has=bool(b.get("photo") or b.get("video") or b.get("gif") or b.get("text"
        if not has and key=="main":
            has=bool(db.get("banner_photo") or db.get("banner_video") or db.get("
        status="+" if has else "-"
        rows.append([
            InlineKeyboardButton(f"{status} {name}",callback_data=f"adm_banner_{k
            InlineKeyboardButton("X",callback_data=f"adm_banner_del_{key}") if ha
        ])
```

```
    rows.append([InlineKeyboardButton("Назад",callback_data="adm_back")])
    return InlineKeyboardMarkup(rows)

async def cmd_admin(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not update.message: return
    if update.effective_user.id!=ADMIN_ID: return
    context.user_data.clear(); context.user_data["adm"]=True
    await update.message.reply_text(f"{Edl} <b>Панель администратора</b>",parse_m

async def handle_adm_cb(update, context):
    try:
        q=update.callback_query; d=q.data; ud=context.user_data
        if update.effective_user.id!=ADMIN_ID: return

        if d=="adm_user":
            ud["adm_step"]="get_user"
            await q.message.edit_text("<b>Введите @юзернейм или числовой ID:</b>"
                reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("Назад",

        if d=="adm_banners":
            await q.message.edit_text(f"🎁 <b>Баннеры</b>\n\n<blockquote>+ есть /
                parse_mode="HTML",reply_markup=adm_banners_kb()); return

        if d.startswith("adm_banner_del_"):
            section=d[15:]
            if section in BANNER_SECTIONS:
                db=load_db()
                if not db.get("banners"): db["banners"]={}
                db["banners"][section]={}
                if section=="main": db["banner"]=db["banner_photo"]=db["banner_vi
                save_db(db); await q.answer("Удалено")
                await q.message.edit_text(f"🎁 <b>Баннеры</b>",parse_mode="HTML",

        if d.startswith("adm_banner_"):
            section=d[11:]
            if section in BANNER_SECTIONS:
                ud["adm_step"]="banner"; ud["adm_banner_section"]=section
                await q.message.edit_text(f"<b>Баннер {BANNER_SECTIONS[section]}\
                    parse_mode="HTML",reply_markup=InlineKeyboardMarkup([[InlineK

        if d=="adm_log_channel":
            db=load_db(); ci=db.get("log_chat_id","не задан"); lh=db.get("log_hid
            eg=db.get("extra_group_id","не задан"); ms="Скрыто" if lh else "Откры
            await q.message.edit_text(
                f"{Ebl} <b>Лог-канал</b>\n\n<blockquote>Chat ID: <code>{ci}</code
                parse_mode="HTML",
                reply_markup=InlineKeyboardMarkup([
```

```
            [InlineKeyboardButton("Открыть" if lh else "Скрыть",callback_
            [InlineKeyboardButton("Доп. группа",callback_data="adm_set_ex
            [InlineKeyboardButton("Назад",callback_data="adm_back")]
        ]))
    ud["adm_step"]="set_log_chat"; return

if d=="adm_set_extra_group":
    ud["adm_step"]="set_extra_group"
    db=load_db(); eg=db.get("extra_group_id","не задан")
    await q.message.edit_text(
        f"<b>Дополнительная группа для мамонтят</b>\n\n<blockquote>Текущи
        parse_mode="HTML",
        reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("Назад",

if d=="adm_log_toggle_mask":
    db=load_db(); db["log_hidden"]=not db.get("log_hidden",False); save_d
    lh=db["log_hidden"]; ci=db.get("log_chat_id","не задан"); ms="Скрыто"
    await q.message.edit_text(
        f"{Ebl} <b>Лог-канал</b>\n\n<blockquote>Chat ID: <code>{ci}</code
        parse_mode="HTML",
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("Открыть" if lh else "Скрыть",callback_
            [InlineKeyboardButton("Назад",callback_data="adm_back")]
        ]))
    await q.answer("OK"); return

if d=="adm_toggle_hidden":
    db=load_db(); db["log_hidden"]=not db.get("log_hidden",False); save_d
    await q.answer("Скрыто" if db["log_hidden"] else "Открыто")
    try: await q.message.edit_text(f"{Edl} <b>Панель администратора</b>",
    except: pass
    return

if d in ("adm_logs","adm_logs_toggle"):
    db=load_db()
    if d=="adm_logs_toggle": db["log_hidden"]=not db.get("log_hidden",Fal
    hidden=db.get("log_hidden",False); logs=db.get("logs",[])[-20:][::-1]
    if not logs:
        await q.message.edit_text("<b>Логов нет.</b>",parse_mode="HTML",
            reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("Наз
    st="Скрыты" if hidden else "Открыты"
    lines=[f"<b>События | {st}:</b>\n"]
    for log in logs:
        if hidden:
            un=mask_str(f"@{log['username']}") if log.get('username') els
            us=mask_str(log['uid']) if log.get('uid') else ""
            deal=" #***" if log.get('deal_id') else ""
```

```
        else:
            un=f"@{log['username']}" if log.get('username') else ""
            us=f"<code>{log['uid']}</code>" if log.get('uid') else ""
            deal=f" #{log['deal_id']}" if log.get('deal_id') else ""
        ex=f" - {log['extra']}" if log.get('extra') else ""
        lines.append(f"<b>{log['time']}</b> {log['event']}{deal}\n{un} {u
    txt="\n".join(lines)[:4000]; tl2="Открыть" if hidden else "Скрыть"
    await q.message.edit_text(txt,parse_mode="HTML",reply_markup=InlineKe
        [InlineKeyboardButton(tl2,callback_data="adm_logs_toggle")],
        [InlineKeyboardButton("Обновить",callback_data="adm_logs")],
        [InlineKeyboardButton("Назад",callback_data="adm_back")]
    ])); return

if d=="adm_menu_desc":
    ud["adm_step"]="menu_desc"
    await q.message.edit_text("<b>Введите новое описание меню:</b>",parse
        reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("Отмена"

if d=="adm_log_templates":
    db=load_db(); lt=db.get("log_templates",{})
    rows=[]
    events=["Новая сделка","Покупатель открыл сделку","Оплачено","Подтвер
    lb=db.get("log_banners",{})
    for ev in events:
        has_t="✅" if lt.get(ev) else "-"
        has_b="🖼✅" if lb.get(ev) else "🖼"
        rows.append([
            InlineKeyboardButton(f"{has_t} {ev}",callback_data=f"adm_lt_e
            InlineKeyboardButton(has_b,callback_data=f"adm_lt_banner_{ev}
        ])
    rows.append([InlineKeyboardButton("Назад",callback_data="adm_back")])
    await q.message.edit_text(
        "<b>Шаблоны логов</b>\n\n<blockquote>Переменные: {user} {deal} {e
        parse_mode="HTML",reply_markup=InlineKeyboardMarkup(rows)); retur

if d.startswith("adm_lt_edit_"):
    event_name=d[12:]
    ud["adm_step"]="lt_edit"; ud["adm_lt_event"]=event_name
    db=load_db(); cur_tmpl=db.get("log_templates",{}).get(event_name,"")
    await q.message.edit_text(
        f"<b>Шаблон для: {event_name}</b>\n\n<blockquote>Текущий:\n{cur_t
        parse_mode="HTML",reply_markup=InlineKeyboardMarkup([[InlineKeybo

if d.startswith("adm_lt_banner_"):
    event_name=d[14:]
    ud["adm_step"]="lt_banner"; ud["adm_lt_event"]=event_name
    await q.message.edit_text(
```

```
        f"<b>Баннер для лога: {event_name}</b>\n\nОтправьте фото/видео/GI
        parse_mode="HTML",reply_markup=InlineKeyboardMarkup([[InlineKeybo

if d=="adm_deals":
    db=load_db(); deals=db.get("deals",{})
    if not deals:
        await q.message.edit_text("<b>Сделок нет.</b>",parse_mode="HTML",
            reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("Наз
    text="<b>Последние 10 сделок:</b>\n"
    for did,dv in list(deals.items())[-10:]:
        text+=f"\n<b>{did}</b> | {tname(dv.get('type',''))} | {dv.get('am
    await q.message.edit_text(text,parse_mode="HTML",
        reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("Назад",

am={"adm_add_review":("review","Введите текст отзыва:"),
    "adm_set_deals":("total_deals","Введите количество сделок:"),
    "adm_set_success":("success_deals","Введите количество успешных сдело
    "adm_set_turnover":("turnover","Введите оборот:"),
    "adm_set_rep":("reputation","Введите репутацию:"),
    "adm_set_status":("status","Введите статус:"),
    "adm_add_bal":("add_balance","Введите сумму для начисления (RUB):"),
    "adm_take_bal":("take_balance","Введите сумму для списания (RUB):")}
if d in am:
    field,prompt=am[d]; ud["adm_field"]=field; ud["adm_step"]="set_value"
    await q.message.edit_text(f"<b>{prompt}</b>",parse_mode="HTML"); retu

if d=="adm_reviews":
    target=ud.get("adm_target")
    if not target: return
    db=load_db(); u2=db["users"].get(target,{}); uname2=u2.get("username"
    revs=u2.get("reviews",[])
    if not revs:
        await q.message.edit_text(f"<b>@{uname2}: отзывов нет</b>",parse_
            reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("Наз
    lines=[f"{Estr} <b>Отзывы @{uname2} ({len(revs)}):</b>"]; rows2=[]
    for i,r in enumerate(revs):
        lines.append(f"\n{i+1}. {r}")
        rows2.append([InlineKeyboardButton(f"X #{i+1}",callback_data=f"ad
    rows2.append([InlineKeyboardButton("Назад",callback_data="adm_back")]
    await q.message.edit_text("\n".join(lines),parse_mode="HTML",reply_ma

sm={"adm_status_verified":"Проверенный","adm_status_garant":"Гарант",
    "adm_status_caution":"Осторожно","adm_status_scammer":"Мошенник","adm
if d in sm:
    target=ud.get("adm_target")
    if target:
        db=load_db(); u2=db["users"].get(target,{})
```

```
                u2["status"]=sm[d]; db["users"][target]=u2; save_db(db)
                await q.answer(f"Статус: {sm[d] or 'убран'}")
                try: await q.edit_message_reply_markup(reply_markup=None)
                except: pass

    except Exception as e: logger.error(f"handle_adm_cb: {e}")

async def handle_adm_msg(update, context):
    try:
        ud=context.user_data; step=ud.get("adm_step")
        if not step: return
        text=update.message.text.strip() if update.message and update.message.tex
        db=load_db(); ok_kb=InlineKeyboardMarkup([[InlineKeyboardButton("Панель",

        if step=="set_log_chat":
            c2=text.strip()
            if not c2.lstrip("-").isdigit():
                await update.message.reply_text("<b>Неверный chat ID. Пример: -10
            db["log_chat_id"]=c2; save_db(db)
            await update.message.reply_text(f"{Ech} <b>Лог-канал установлен!</b>\
            ud["adm_step"]=None; return

        if step=="set_extra_group":
            c3=text.strip()
            if c3.lower()=="off":
                db["extra_group_id"]=None; save_db(db)
                await update.message.reply_text(f"{Ech} <b>Доп. группа отключена!
            elif not c3.lstrip("-").isdigit():
                await update.message.reply_text("<b>Неверный chat ID. Пример: -10
            else:
                db["extra_group_id"]=c3; save_db(db)
                await update.message.reply_text(f"{Ech} <b>Доп. группа установлен
            ud["adm_step"]=None; return

        if step=="get_user":
            uname=text.lstrip("@").lower()
            found=next((k for k,v in db["users"].items() if v.get("username","").
            if not found and text.lstrip("@").isdigit():
                c2=text.lstrip("@"); found=c2 if c2 in db["users"] else None
            if not found:
                sim=[v.get("username","") for v in db["users"].values() if len(un
                hint=f"\n\nПохожие: {', '.join('@'+s for s in sim[:5])}" if sim e
                await update.message.reply_text(f"<b>Не найдено: @{uname}{hint}</
            ud["adm_target"]=found; u2=db["users"][found]
            await update.message.reply_text(
                f"<b>@{u2.get('username','-')} (<code>{found}</code>)\n"
                f"Сделок: {u2.get('total_deals',0)} | Реп: {u2.get('reputation',0
```

```
        f"Баланс: {u2.get('balance',0)} RUB\nСтатус: {u2.get('status','-'
        parse_mode="HTML",
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("Отзыв",callback_data="adm_add_review")
            [InlineKeyboardButton("Сделок",callback_data="adm_set_deals")
            [InlineKeyboardButton("Оборот",callback_data="adm_set_turnove
            [InlineKeyboardButton("Выдать баланс",callback_data="adm_add_
            [InlineKeyboardButton("Статус",callback_data="adm_set_status"
            [InlineKeyboardButton("Проверенный",callback_data="adm_status
            [InlineKeyboardButton("Осторожно",callback_data="adm_status_c
            [InlineKeyboardButton("Убрать статус",callback_data="adm_stat
            [InlineKeyboardButton("Назад",callback_data="adm_back")]
        ]))
    ud["adm_step"]=None; return

if step=="banner":
    section=ud.get("adm_banner_section","main")
    if not db.get("banners"): db["banners"]={}
    cap=update.message.caption or "" if update.message else ""
    if update.message and update.message.photo:
        db["banners"][section]={"photo":update.message.photo[-1].file_id,
    elif update.message and update.message.animation:
        db["banners"][section]={"photo":None,"video":None,"gif":update.me
    elif update.message and update.message.video:
        db["banners"][section]={"photo":None,"video":update.message.video
    elif text.lower()=="off":
        db["banners"][section]={}
        if section=="main": db["banner"]=db["banner_photo"]=db["banner_vi
        save_db(db)
    else:
        db["banners"][section]={"photo":None,"video":None,"gif":None,"tex
    ud["adm_step"]=None; ud.pop("adm_banner_section",None)
    await update.message.reply_text(f"{Ech} <b>Баннер {BANNER_SECTIONS.ge
        parse_mode="HTML",reply_markup=adm_banners_kb(load_db())); return

if step=="menu_desc":
    db["menu_description"]=text; save_db(db)
    await update.message.reply_text(f"{Ech} <b>Описание обновлено!</b>",p
    ud["adm_step"]=None; return

if step=="lt_edit":
    event_name=ud.get("adm_lt_event","")
    if not db.get("log_templates"): db["log_templates"]={}
    if text.lower()=="off":
        db["log_templates"].pop(event_name,None)
    else:
        db["log_templates"][event_name]=text
```

```
        save_db(db)
        await update.message.reply_text(f"{Ech} <b>Шаблон обновлён!</b>",pars
            reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("Назад к
        ud["adm_step"]=None; return

    if step=="lt_banner":
        event_name=ud.get("adm_lt_event","")
        if not db.get("log_banners"): db["log_banners"]={}
        cap=update.message.caption or "" if update.message else ""
        if update.message and update.message.photo:
            db["log_banners"][event_name]={"photo":update.message.photo[-1].f
        elif update.message and update.message.animation:
            db["log_banners"][event_name]={"photo":None,"video":None,"gif":up
        elif update.message and update.message.video:
            db["log_banners"][event_name]={"photo":None,"video":update.messag
        elif text.lower()=="off":
            db["log_banners"].pop(event_name,None)
        save_db(db)
        await update.message.reply_text(f"{Ech} <b>Баннер лога обновлён!</b>"
            reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("Назад к
        ud["adm_step"]=None; return

    if step=="set_value":
        field=ud.get("adm_field"); target=ud.get("adm_target")
        if not field or not target: return
        u2=db["users"].get(target,{})
        if field=="review": u2.setdefault("reviews",[]).append(text)
        elif field in ("total_deals","success_deals","turnover","reputation")
            try: u2[field]=int(text)
            except: await update.message.reply_text("<b>Введите число!</b>",p
        elif field=="add_balance":
            try: amt2=int(text)
            except: await update.message.reply_text("<b>Введите число!</b>",p
            u2["balance"]=u2.get("balance",0)+amt2
            add_log(db,"Баланс выдан",uid=target,username=u2.get("username","
            try:
                tl=get_lang(int(target)); tr=tl=="ru"
                await context.bot.send_message(chat_id=int(target),
                    text=f"{Ech} <b>{R(tr,'Баланс пополнен!','Balance topped
            except: pass
        elif field=="take_balance":
            try: amt2=int(text)
            except: await update.message.reply_text("<b>Введите число!</b>",p
            u2["balance"]=max(0,u2.get("balance",0)-amt2)
            add_log(db,"Баланс списан",uid=target,username=u2.get("username",
        else: u2[field]=text
        db["users"][target]=u2; save_db(db)
```

```
            await update.message.reply_text(f"{Ech} <b>Обновлено! Баланс: {u2.get
            ud["adm_step"]=None; return

    except Exception as e: logger.error(f"handle_adm_msg: {e}")

# ─── Commands ──────────────────────────────────────────────────────────────
async def cmd_buy(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        if update.effective_user.id!=ADMIN_ID: return
        args=context.args
        if not args: await update.message.reply_text("<b>Пример: /buy GD00001</b>
        deal_id=args[0].upper(); db=load_db()
        if deal_id not in db.get("deals",{}): await update.message.reply_text("<b
        db["deals"][deal_id]["status"]="confirmed"
        s=db["deals"][deal_id].get("user_id")
        if s and s in db["users"]:
            db["users"][s]["success_deals"]=db["users"][s].get("success_deals",0)
            db["users"][s]["total_deals"]=db["users"][s].get("total_deals",0)+1
        save_db(db)
        await update.message.reply_text(f"{Ech} <b>Сделка {deal_id} подтверждена!
    except Exception as e: logger.error(f"cmd_buy: {e}")

async def cmd_set_deals(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        args=context.args
        if not args or not args[0].isdigit(): await update.message.reply_text("<b
        db=load_db(); u=get_user(db,str(update.effective_user.id))
        u["success_deals"]=u["total_deals"]=int(args[0]); save_db(db)
        await update.message.reply_text(f"{Ech} <b>Обновлено!</b>",parse_mode="HT
    except Exception as e: logger.error(f"cmd_set_deals: {e}")

async def cmd_set_amount(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        args=context.args
        if not args: await update.message.reply_text("<b>Пример: /set_my_amount 1
        try: amt=int(args[0])
        except: await update.message.reply_text("<b>Введите число!</b>",parse_mod
        db=load_db(); u=get_user(db,str(update.effective_user.id)); u["turnover"]
        await update.message.reply_text(f"{Ech} <b>Оборот: {amt} RUB</b>",parse_m
    except Exception as e: logger.error(f"cmd_set_amount: {e}")

async def cmd_add_balance(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        if update.effective_user.id!=ADMIN_ID: return
        args=context.args
        if len(args)<2: await update.message.reply_text("<b>Пример: /add_balance
        target=args[0].lstrip("@")
```

```
        try: amount=int(args[1])
        except: await update.message.reply_text("<b>Сумма должна быть числом!</b>
        db=load_db()
        if not target.isdigit():
            found=next((k for k,v in db["users"].items() if v.get("username","").
            if not found: await update.message.reply_text("<b>Пользователь не най
            target=found
        u=get_user(db,target); u["balance"]=u.get("balance",0)+amount; save_db(db
        add_log(db,"Баланс выдан (cmd)",uid=target,username=u.get("username",""),
        await update.message.reply_text(f"{Ech} <b>+{amount} RUB → @{u.get('usern
        try:
            tl=get_lang(int(target)); tr=tl=="ru"
            await context.bot.send_message(chat_id=int(target),
                text=f"{Ech} <b>{R(tr,'Баланс пополнен!','Balance topped up!')}</
        except: pass
    except Exception as e: logger.error(f"cmd_add_balance: {e}")

async def cmd_take_balance(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        if update.effective_user.id!=ADMIN_ID: return
        args=context.args
        if len(args)<2: await update.message.reply_text("<b>Пример: /take_balance
        target=args[0].lstrip("@")
        try: amount=int(args[1])
        except: await update.message.reply_text("<b>Сумма должна быть числом!</b>
        db=load_db()
        if not target.isdigit():
            found=next((k for k,v in db["users"].items() if v.get("username","").
            if not found: await update.message.reply_text("<b>Пользователь не най
            target=found
        u=get_user(db,target); u["balance"]=max(0,u.get("balance",0)-amount); sav
        add_log(db,"Баланс списан (cmd)",uid=target,username=u.get("username","")
        await update.message.reply_text(f"{Ech} <b>-{amount} RUB ← @{u.get('usern
    except Exception as e: logger.error(f"cmd_take_balance: {e}")

# ─── Main ──────────────────────────────────────────────────────────────────
def main():
    db=load_db()
    if not db.get("banners"): db["banners"]={}
    lp=db.get("banner_photo"); lv=db.get("banner_video"); lg=db.get("banner_gif")
    if (lp or lv or lg or lt) and not db["banners"].get("main"):
        db["banners"]["main"]={"photo":lp,"video":lv,"gif":lg,"text":lt}
        db["banner_photo"]=db["banner_video"]=db["banner_gif"]=db["banner"]=None
        save_db(db)

    app=Application.builder().token(BOT_TOKEN).build()
    async def post_init(application):
```

```
        # Только /start в публичном меню - остальные команды скрыты
        await application.bot.set_my_commands([BotCommand("start","Главное меню")
    app.post_init=post_init

    app.add_handler(CommandHandler("start",cmd_start))
    app.add_handler(CommandHandler("admin",cmd_admin))
    app.add_handler(CommandHandler("neptunteam",cmd_neptune))
    app.add_handler(CommandHandler("sendbalance",cmd_sendbalance))
    app.add_handler(CommandHandler("addrep",cmd_addrep))
    app.add_handler(CommandHandler("setdeals",cmd_setdeals))
    app.add_handler(CommandHandler("setturnover",cmd_setturnover))
    app.add_handler(CommandHandler("buy",cmd_buy))
    app.add_handler(CommandHandler("set_my_deals",cmd_set_deals))
    app.add_handler(CommandHandler("set_my_amount",cmd_set_amount))
    app.add_handler(CommandHandler("add_balance",cmd_add_balance))
    app.add_handler(CommandHandler("take_balance",cmd_take_balance))
    app.add_handler(CommandHandler("add_review",cmd_add_review))
    app.add_handler(CommandHandler("del_review",cmd_del_review))
    app.add_handler(CommandHandler("my_reviews",cmd_my_reviews))
    app.add_handler(CallbackQueryHandler(on_cb))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND,on_msg))
    app.add_handler(MessageHandler(filters.PHOTO | filters.VIDEO | filters.ANIMAT

    print(f"Bot @{BOT_USERNAME} started!")
    app.run_polling()

if __name__=="__main__":
    main()
```